Lancaster University

Master of Science in Data Science
School of Computing and Communication

Image Recognition and Deep Learning for COVID-19 CT scans

Supervisor:
Alex Gibberd

Author:
Paschalis Itsios
ID: 35193390

September 2020

Abstract

Automatic detection and prediction of COVID-19 using CT-scans is crucial for dealing with the new pandemic. In this work, we present a Deep Learning framework that can support doctors in their decision-making process. Specifically, we provide an end-to-end model framework which can preprocess, analyse and predict the disease status of patients without wasting significant computational resources. Transfer Learning assists us in training the models with a relatively small medical dataset. Additionally, we attempt to shed light on the black-box mechanism of Convolutional Neural Networks by utilising dimensionality reduction (PCA) and Clustering (GMM). Our best model predicts with 99% accuracy the event that someone is a COVID-19 patient or not. Our framework reduces the dimensions of the raw images and provides representative clustering results. Extracting prototypes and constructing mean images can help the medical practitioners to understand which parts of the images are strong indicators that someone has COVID-19.

Contents

1 Introduction **3**
1.1 Motivation: The COVID-19 Pandemic under the scope of Explainable AI 3
1.2 Aims and objectives 3
1.3 Research questions 4
1.4 Structure of the thesis 4

2 Background and Related work **6**
2.1 Types of Learning. Supervised and unsupervised learning. 6
2.2 Medical imaging, deep learning and transfer learning 6
2.3 Introduction to Neural networks 7
2.3.1 Perceptron 7
2.3.2 Multi Layer Perceptrons (MLP) 8
2.3.3 Loss and activation functions 8
2.3.4 Training and optimisation 10
2.3.5 Optimisers 11
2.3.6 Synopsis and softmax classification 12
2.4 Convolutional neural networks and Transfer learning 13
2.4.1 Understanding the convolution 13
2.4.2 Padding and Striding 14
2.4.3 Pooling 15
2.4.4 ConvNets architectures and VGG16 15
2.4.5 Efficiency through transfer learning. 16

3 Data **18**
3.1 Data sources and pre-processing 18
3.1.1 Data pre-processing and scaling 19

4 Classification using CNN **20**
4.1 Description of the proposed model 20
4.1.1 First method: Final Layer Transfer Learning with VGG16 21
4.1.2 Second method: Five last Layers Transfer Learning with VGG16 22
4.1.3 Third method: Mid-layers Transfer Learning with VGG16 23
4.2 Cross-validation and performance evaluation 26

5 Understanding Neural Network Activation **28**
5.1 Feature extraction and Dimensionality reduction 28
5.2 Clustering using Gaussian Mixture Models 29

6 Discussion **39**

7 Conclusion **41**
7.1 Future work 41

References **42**

8 Appendix A **45**
8.1 Project specification 45

1 Introduction

1.1 Motivation: The COVID-19 Pandemic under the scope of Explainable AI

The role of Artificial Intelligence (AI) on fighting against new diseases is becoming more central nowadays after the explosion of the COVID-19 pandemic[Vaishya et al., 2020]. So the changes that are taking place into this field are enormous. This work focuses on trying to apply deep-learning methods to help potentially by diagnosing and understanding COVID-19 infections.

On 31st December 2019, the World Health Organization (WHO) was informed about some sporadic cases of pneumonia in Wuhan City in China. The WHO, after examination assessed the cause of those cases, identified this as a novel coronavirus and named it "2019-ncOv". According to the European Surveillance System [TES] COVID-19 is a viral infection, and histologic findings from lungs examinations indicate that the damage from COVID-19 is common with other respiratory viruses like MERS-Cov, common cold and influenza virus. However, some findings can distinguish COVID-19 from aforementioned diseases. Generally, the damage of the vascular cells and the appearance of thrombosis are common in COVID-19 cases [Ackermann et al., 2020]. On 11th March 2020, the severity of the situation forced the WHO to declare the outbreak a pandemic.

For this reason, different research domains have strengthen their efforts towards the development of a sustainable mechanism which can tackle the arisen obstacles. Data scientists are focused on assisting medical practitioners with a complementary tool because there is only a restricted number of available test kits. Many researchers apply deep learning methodologies to detect COVID-19 patients by analysing the available CT scans or Xray images of their lungs [Nguyen, 2020]. These images are processed and converted into visual representations through medical imaging. Also Alimadadi et al. [2020] proposed a machine learning framework which uses real-time data to understand the pattern behind the spread of the virus and to identify patients which are vulnerable to COVID-19 based on some genetic characteristics of them. Some areas of interest that medical imaging deals with are the image quality evaluation in colonoscopy, the computer tomography (CT) scans for pulmonary diseases and the brain tumour segmentation in magnetic resonance imaging (MRI) [Tajbakhsh et al., 2016]. In our research, medical imaging is combined with deep learning, and efficient results are generated.

The data regarding the COVID-19 in the first term of 2020 was enormous. However, this data contains much information which has not been utilised yet. Our research was conducted to address the problem of the redundancy of this medical data, and instead, use it to predict outcomes of COVID-19 patients. Our study aims to produce insights and as a result, to provide a framework that can transform the black-box mechanism of deep learning models into something more explainable.

1.2 Aims and objectives

The first aim is to construct a pipeline which has the potential to preprocess and analyse COVID-19 CT scan images but also predict the event that a new patient is detected positive to COVID-19, by training models using their CT scans. Various authors suggest methods to extract features from convolutional neural networks and classify the abnormalities of different medical respiratory syndromes, including COVID-19 [Khuzani et al., 2020]. First, convolutional neural networks look to be used, and our objective is to experiment with a simple model architecture, such as VGG16, that can be easily trained. Next, transfer learning and fine-tuning could potentially enhance the computational efficiency and predictive performance of our model because of two reasons. The first one is that transfer learning uses less computational resources because the trainable parameters for a model which utilised transfer learning are less than the parameters of a model which is trained from scratch. The second one is that it uses prior knowledge from an already pre-trained model to initialise the weights rather than using random weight initialisation, and this could lead quickly to better predictive performance.

The second aim of this work is to explain the decisions made by neural network classifiers. As such, we will examine dimensionality reduction and clustering techniques in order to understand how

labels align with the clusters identified in the low dimensional representations. The motivation behind our effort to interpret the generated results is because neural networks are considered as black-boxes [Buhrmester et al., 2019]. So it is of high importance to produce some results that could be understood by people who are not data scientists or who do not have the medical domain expertise. In other words, we aim to find what is behind the decision making procedure which took place by the mechanism of Neural Networks.

The contribution of this work can be summarised into the following three points:

- Three proposed models that utilised deep learning and detected new COVID-19 patients by looking at CT-scans of their lungs.
- A scrutinised examination of how different model variations could help medical diagnosis by improving the doctors' decision-making process.
- An efficient and explainable framework that does not require exhaustive computational resources in order to be utilised.

1.3 Research questions

The questions we aim to answer are:

1. Does deep learning provide a reliable tool for the prediction of COVID-19 patients?
2. Is transfer learning the right approach, both in terms of computational efficiency and performance?
3. Could fine-tuning enhance the performance of the model?
4. Can we explain the decision-making process of a deep neural network by finding discrete clusters of COVID and non-COVID samples and by constructing mean images which contain the most characteristic features of each cluster?
5. Are the features that were extracted from lower-level layers better both in terms of performance and interpretability ?

1.4 Structure of the thesis

This work is organised to simulate the process of the data science pipeline. We started by identifying the gap in the literature regarding the novel COVID-19 outbreak and understanding the problem. Next, we constructed a set of research questions which could be tackled within the constraints of the project. In section 3 we present how the data sources were gathered and preprocessed. The next chapter encloses the background research and related work which had been conducted by other researchers, and it is described how their work could be utilised in our research. In Section 4 and Section 5, the framework of our methodology is provided, and the data modelling procedure is presented.

Specifically in Section 4, we deployed three different variations of the VGG16 model, which yielded promising results. That is, we modified the VGG16 architecture and also we utilised the transfer learning mechanism in order to classify images and to predict the classes of the unseen data. The three suggested CNN's were first pre-trained in Imagenet and next they were accordingly engineered in order to adapt to our dataset. The models' performance was assessed by using different performance metrics, and the results of each model were compared with the other models. Also, fine-tuning was considered. That is, we trained the whole network with a lower learning rate, and this yielded an enhancement to the model performance.

In section 5, Principal Components Analysis (PCA) was implemented to the features of the second fully-connected layer for each one of our three proposed models. The most important features which explained 90% of the variance kept and clustering techniques were used as a tool for more explainable

results. To further support our analysis, we utilised Gaussian Mixture Models to provide a probabilistic model which can cluster the data. The most representative image (prototype) of each cluster was extracted. After, the mean image of each cluster was estimated by taking a weighted probability sum of the image values. The images were weighted based on the conditional probabilities that soft assignment of GMM achieved. We used the z matrix that was generated by the GMM clustering. This matrix contained the conditional probabilities of belonging to one of the different clusters and every image was assigned with different cluster assignments. Potentially, those constructed images would assist the medical researchers and practitioners to understand which are the specific patterns that may imply that someone is a COVID patient or not.

Also, the results of our experiments were evaluated, and appropriate comparisons were conducted. A discussion of our main findings and future work are presented in the last part of this thesis.

2 Background and Related work

In this section, we discuss the related work of this dissertation. The first part introduces the reader to the notion of deep learning and how it contributes to medical imaging. The second part describes the theory behind Neural Network (NN) models. The third part describes the theory behind the Convolutional Neural Networks (CNN) and explains the VGG16 architecture. The last part is an effort to provide related work around Transfer Learning and to show how it is implemented on medical imaging.

2.1 Types of Learning. Supervised and unsupervised learning.

According to Lecun et al. [2015] the background of the Deep Learning was set a couple of decades ago but efficient feature extraction used to be a difficult task for the conventional algorithms that were used back then. What deep learning brought to the scientific society is a set of methods that let scientists feed the machines with raw data and get as feedback, insightful results. Essentially, representation learning is the breakthrough of the deep-learning methods, and it can be determined as the automatic method which reveals patterns going from lower to higher representations and classifies between different labels using non-linear compositions of modules. These models are of high importance when the research objective is about image analysis. One characteristic of deep learning is that the model is trained by using general-purpose mechanisms.

The types of learning can be distinguished into two broad categories. The first type of learning is *Supervised learning*, and it is examined in the first part of our methodology in section 4. The model receives the data which are aligned to different classes. Essentially each class is a distinct category. Each data point consists of different features and labels. The algorithm tries to generate a vector of scores that align the examined data point to its real class. We want the score of one category to be as high as possible for the desired category. That said, the algorithm receives as an input different images and their real categories. The training of the model is conducted and the objective is to minimise the error.[Lecun et al., 2015] The training mechanism is described more thoroughly at section 2.3.4.

The second type of learning is *Unsupervised Learning*, and it is analysed in the second part of our methodology in section 5. The way that human learns can be compared with the Unsupervised learning in the sense that we learn the world around us by observing it and not by having prior knowledge regarding the labels of the different objects [Lecun et al., 2015]. Hence, under the context of Unsupervised Learning, the training data consists of the features, while the target values are not provided. Discovering the patterns of the data can help us to understand them. This can be done by grouping data that have common characteristics and similarities. Clustering and dimensionality reduction are some well-known techniques that can help us conduct the Unsupervised Learning, and their implementation is presented in section 5.

2.2 Medical imaging, deep learning and transfer learning

The explosion of the recent COVID pandemic addresses the importance of a stable healthcare information technology (IT) system, which can provide the healthcare experts with real-time assistance. Pryor et al. [2020] highlighted the need for an efficient and well-designed prevention tool on the hands of medical practitioners in order to identify possible active patients quickly and taking measures in advance. Hence, the need for optimising healthcare platforms seems to be of high importance when it comes to the prevention of pandemic's spread. Technology can potentially help stabilise the healthcare infrastructure during the pandemic period.

One of the most critical aspects of this work is to examine how technology, machine learning and artificial intelligence, could cooperate with medical imaging and producing valuable insights for diagnosing respiratory infections. A significant pillar of machine learning is deep learning which provides enormous capabilities to a wide range of fields, including the medical sector. One widely used class of models in medical image analysis is the Convolutional Neural Networks (CNNs/ConvNets)[Oquab

et al., 2014]. Also, the usage of transfer learning can be considered a breakthrough in Deep Learning because the last decades the evolution of big data and machine learning set a twofold priority; developing more accurate models, and maximising computational efficiency.

There is much research conducted regarding the transfer learning in medical imaging, and one crucial question raised is what are the benefits of training a model from scratch instead of utilising transfer learning. Tajbakhsh et al. [2016] tried to answer this question by experimenting across a variety of medical imaging fields. Specifically, they compared the performance of models that were trained from scratch and models created by using transfer learning and fine-tuning at their modelling procedure. Ultimately, [Tajbakhsh et al., 2016] summarised that fine-tuning different stages of a network incrementally could accomplish some extraordinary results. That is, they first fine-tuned only the last layer before the output layer, and they continued incrementally by fine-tuning all the high-level layers which enclose some specific pattern for the target dataset, so their contribution to the learning procedure is of high importance. However, they highlighted through empirical experiments the importance of training deeper layers in the case of a high difference between the source and the target task.

The work of Tajbakhsh et al. [2016] can be distinguished from the work of Bar et al. [2015] in the sense that the latter research was focused on using pre-trained models, extracting their features from a fully-connected layer and feeding them into a new classifier, while the former aimed to present a compact model which was boosted by using fine-tuning in a layer-wise manner. Also, one other approach which had good results was adopted by Carneiro et al. [2015] who proposed a procedure of transfer learning by adapting the last classification layer into the needs of the target problem (the one we want to solve). Hence, they removed the final fully-connected layer, and they trained a new one with their training dataset. Furthermore, they showed that the classification results of the pre-trained model were much better compared to the results of a model which was randomly initialised.

2.3 Introduction to Neural networks

The notion of Neural Networks was introduced in 1976 by Ivakhnenko et al. [1967], who presented a deep network with eight layers. One significant achievement of Deep Learning was IBM's supercomputer which was able to beat the world champion, Garry Kasparov, in a chess game in 1996. The human brain subconsciously perceives the external information but the mechanism behind it is not apparent. Many scientists tried to inherit this "skill" to the machines through the mechanism of perception.

2.3.1 Perceptron

Neural networks are partially inspired by the nervous system in the brain, and they were used initially as a mechanism to describe how the human brain neurons are connected [Lindsay, 2020]. Neurons are the functional units of the neural networks, and the connections between them called synapses. The inputs $X = [x_1, x_2 \ldots, x_n]$ combined by their weights $W = [w_1, w_2, \ldots, w_n]$ and a bias term (b) as the following equation describes:

$$m = f(W^T X + b) \ . \tag{1}$$

As Equation (1) describes, every input (X) is assigned with weights (W). The weights indicate the power of the connection between neurons. They are combined linearly with the synapses, and this can be represented by the matrix multiplication. Next, the input values are multiplied by the weights, and their sum is calculated. An activation function receives as input the inner product and decides how relevant this input is in order to pass it to the next neuron. The threshold unit is the mechanism which is responsible for deciding how relevant every input is to the classification problem. This mechanism is visualised at the left part in Figure 1. It is the most straightforward neural network representation which is used for binary classification problems.

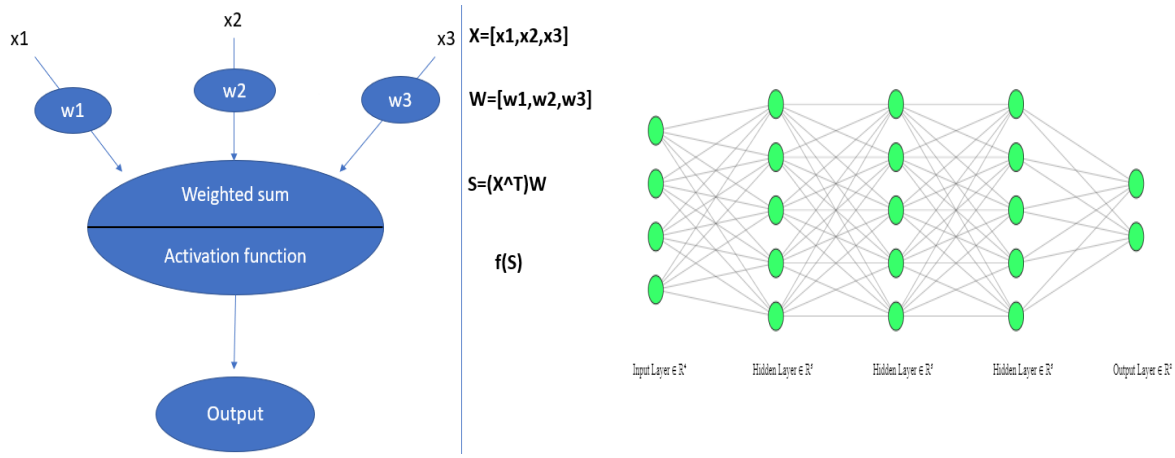


Figure 1: Left:Multiple Layer Perceptrons with 3 nodes. Right: Deep Neural Network with 3 hidden layers.

2.3.2 Multi Layer Perceptrons (MLP)

Essentially, a neural network consists of stacked neurons (perceptrons) which are connected using some activation function. Every neuron has many inputs, but only one output and when they stacked are called layers. Ultimately, the objective is to find the function f which maps the inputs(x_i) to some outputs y_i . This can be expressed mathematically as $y = f(x_i; \phi)$ where x_i is the input, y_i is the output, and ϕ are the parameters of the network. For a given layer of the network, we have an output which is described as $y = f(W^T X + b)$. The weights W can be represented by one mxn dimensional matrix ($W \in R^{mxn}$), where m is the number of the neurons of the previous layer (L-1), and n is the number of the neurons of the current layer (L). So, every neuron of the previous layer (L-1) has m weights assignments in order to connect with all the neurons of the current (L) layer. The output of the activation function represents the input values of the current layer (L) and has $n \times 1$ dimensions. This procedure continues for all the layers of the network. The feed-forward neural network consists of a combination of different functions $f(x) = f^{(3)}(f^{(2)}(f^{(1)}(x; \phi)))$ which are connected and the parameters are updated through the chain rule. The *depth* of the network is essentially defined by the number of hidden layers and in this individual example is equivalent to three(3). A visual representation of a neural network with three hidden layers can be presented in the right part of Figure 1. The higher the number of layers is, the deeper the network becomes. Hence, there are many degrees of freedom, and as a result, the function is more flexible to modelling inputs. The output of each neuron is passed through a non-linear activation function to the next hidden layer. Finally, the network learns through the activation of its neurons when to activate specific regions of the input shape. Backpropagation is performed after the forward pass, and it assists with adjusting the parameters (weights and biases).

2.3.3 Loss and activation functions

An essential concept for designing the neural networks is the *loss function*. Loss function does not return negative values, and the lower the value of the loss function, the better for the described model. Cross-entropy is widely used under the context of neural networks as a loss function, and the objective is to minimize it. This can be achieved when the actual output (y) is close to the predicted output (a). The predicted probability is compared to the actual class (0 or 1), and a different penalty is assigned. The penalty depends on the difference between the prediction probability and the actual value.

We define the cross-entropy loss function for one training instance as:

$$L(\theta; x, y) = -\left[y \ln a + (1 - y) \ln(1 - a)\right], \tag{2}$$

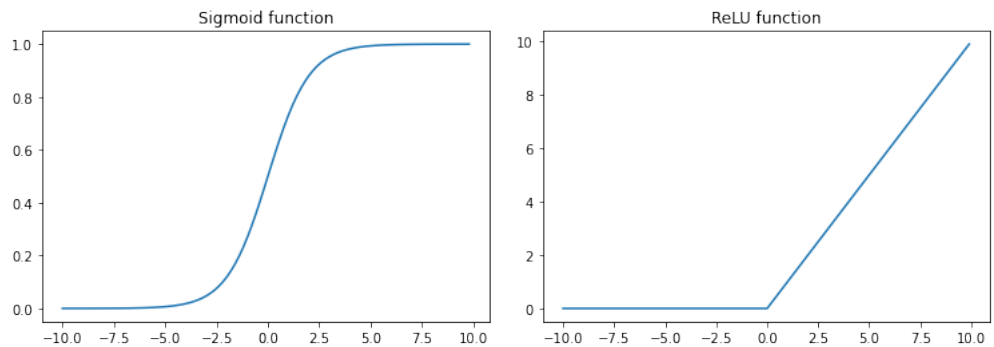


Figure 2: Plots of two activation functions. Left:sigmoid function Right:ReLU funtion

, where n is the number of training inputs, y is the actual output and a is the activation (predicted output).

At this point it is worth to clarify that the cost function is simply the average of the loss-function for all the training samples. This can simply be summarized by the next equation:

$$C(\theta) = \frac{1}{n} \sum_{i=1}^{n} L(\theta; x_i, y_i) \tag{3}$$

where L is the loss function.

To continue with, there is a wide variety of *activation functions*, either linear or non-linear. Some of the are most widely used activation functions are the sigmoid, tahn and RELU. The MLP comes as an extension to the linear and logistic models which cannot capture non-linear functions. Since the 1960's linear classifiers were widely used but issues had arisen due to the sensitivity of those classifiers on different positions or orientation of the input images. his forced scientists to come up with more robust solutions which can use the generic non-linear features and overcome the usage of 'shallow' classifiers. For these reasons scientists came up with non linear transformations of the input $\boldsymbol{x}$, the non-linear function $f(\boldsymbol{x})$, which are used extensively in deep neural networks [Lecun et al., 2015].

The simplest activation function for perceptrons is called step function. When the input is higher than zero(0), then the node is activated while when the input is lower than zero, the neuron is not activated. The next equation returns the output for the X $=[x_1, \ldots, x_n]$ inputs.

$$\mathrm{y} = \sigma(W^T X) \tag{4}$$

This activation function can be used for the simple linear binary classification

$$\sigma(i) = \begin{cases} 1 & \text{if } i > 0 \\ 0 & \text{if } i < 0 \end{cases} \tag{5}$$

Next, one widely-used activation function is the *logistic function* . This function overpasses the issues caused by the step function and empowers the implementation of the gradient descent. The sigmoid function outputs a range of values between zero and one, and it looks like an S-shaped curve. It is non-linear, differentiable and continuous. The logistic function could be expressed mathematically as:

$$\sigma(z) = \frac{1}{1 + e^{-z}} . \tag{6}$$

One other function is the tahn(z):

$$\tanh(\mathrm{z}) = 2\sigma(2\mathrm{z}) - 1 . \tag{7}$$

Tahn function is S-shaped, continuous and differentiable. The output range contains values from -1 to 1, which forces the output of each layer to be centered around zero.

Rectified Linear Unit Function is computationally very efficient, so it becomes the default activation function, and it has a form:

$$ReLU(z) = max(0, z) \ . \tag{8}$$

2.3.4 Training and optimisation

Training the MLP network is conducted by passing the dataset to the network, and the goal is to find the optimal parameters.

First some symbols are defined as:

- L: Last layer
- $L-1$:Second last layer
- y: Actual output
- C: Cost function
- a: Predicted output ($\hat{y}_i$)
- b: Bias

The activation of the previous layer matrix ($a^{[L-1]}$) is multiplied with the weights (w), and the bias b is taken into account.

$$z^{[L]} = w^{[L]} \ a^{[L-1]} + b^{[L]} \ . \tag{9}$$

Next, $z^{[L]}$ is filtered by the activation function (σ), so for the next part of this section, $f(\cdot)$ is replaced by $\sigma(\cdot)$. We have:

$$a^{[L]} = \sigma(z^{[L]}) \ . \tag{10}$$

On a previous stage of this work, the cross-categorical entropy was examined. At this stage we consider that the cost function is the mean square error (MSE), which is given as:

$$C = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - a_i)^2 \ . \tag{11}$$

Next, a special case of equation (11) is examined. Hence, the MSE for one training sample can be expressed as:

$$C = (y - a)^2 \ . \tag{12}$$

We aim to optimise the equation (12) for all the neurons of the network. This cannot be accomplished by changing an element of the cost function directly. The only way to modify it is through the parameters of the weights and the biases, and by changing those to get a better approximation for the error (e.g. decreased error). This could be done through the mechanism of backpropagation. Backpropagation is a widely-used method that updates the weights of a neural network and leads to achieving a decreased error (cost-function) and better predictions. This can be expressed mathematically through the chain rule.

$$\frac{\partial \mathrm{C}}{\partial \mathrm{w}} = \frac{\partial \mathrm{C}}{\partial \mathrm{a}}\frac{\partial \mathrm{a}}{\partial \mathrm{z}}\frac{\partial \mathrm{z}}{\partial \mathrm{w}} = 2(\mathrm{y} - \mathrm{a}^{(\mathrm{L})})\sigma'(\mathrm{z}^{(\mathrm{L})})\mathrm{a}^{(\mathrm{L}-1)} \tag{13}$$

Also, it is important to highlight that while we want to find the parameters which minimize the cost function, this cannot be guaranteed by calculating the gradient of the error function $E(\theta)$ for the different parameters θ and setting it equal to zero as we can see at the next equation:

$$\nabla E(\theta) = 0 \ . \tag{14}$$

The reason is that this function is a non-convex, and there is the risk of trapping to a local minimum. In general, we would need to evaluate all such conditions like equation 14 in order to guarantee that we were at a global minimum, but this is infeasible for a neural network.

For this reason, the iterative update is adopted to solve this issue. This essentially indicates how much we can improve the prediction result by adjusting the weights proportionately to the gradient. The degree that gradient changes the weights called learning rate, and is expressed by the following update rule formula:

$$w_1 = w_0 - r\frac{\partial C}{\partial w_0} \ , \tag{15}$$

where w_1: new weights, w_0: previous weights and r:learning rate.

Generally, parameters θ contain both weights and biases. The update of a parameter is done by subtracting the product of the learning rate and the first derivative (rate of change) of the cost function from the parameter before the update.

In parametric terms this can be written as:

$$\theta_{i+1} = \theta_i - r\frac{\partial C}{\partial \theta} \ , \tag{16}$$

and the update continues for every sample until to reach at the global minima.

At equation (13), we can observe how backpropagation is implemented to update the model only for one step. Below it is described the procedure when the neural network updates the parameters of the three previous layers. As it is clarified the **input** of the layer (**L**) can be calculated after the matrix multiplication of the inputs of the previous layer(**L-1**) by the weights of each neuron of the layer (**L-1**) which are transformed by an activation function. The next example indicates that the input of the final layer $z^{(3)}$ does not depend on the weights of the same layer $w^{(3)}$ but on the outputs of the previous layer, $a^{(2)}$. This procedure continues since the model backpropagates back to the first layer and finally, the access to the weights become feasible. At this case we have:

$$\frac{\partial \mathrm{C}}{\partial \mathrm{w}} = \frac{\partial C}{\partial a^{(3)}}\frac{\partial a^{(3)}}{\partial z^{(3)}}\frac{\partial z^{(3)}}{\partial a^{(2)}}\frac{\partial a^{(2)}}{\partial z^{(2)}}\frac{\partial z^{(2)}}{\partial a^{(1)}}\frac{\partial a^{(1)}}{\partial z^{(1)}}\frac{\partial z^{(1)}}{\partial w^{(1)}} \ . \tag{17}$$

The backpropagation algorithm is used to approximate which neurons are responsible for the prediction error and to help improve the classification prediction after a number of iterations. The weights and biases are adjusted in order to minimise the error. The loss is the quantity we will attempt to minimise during training, so it should represent a measure of success for the task we aim to solve. After some number of iterations, the function starts converging to the optimal point. Hence, neural networks can be used as a classifier through the utilisation of loss function.

This procedure is very crucial and can be summarised with the next steps. First, the algorithm passes forward for every training input, makes a prediction and calculates the error. Afterwards, the reverse procedure takes place (reverse pass) in order to measure the contribution of each neuron at the error and the optimisation of the model is accomplished. Hence, selecting the appropriate optimiser is crucial.

2.3.5 Optimisers

Optimisers define how Neural Networks learn. The training can be described as an iterative procedure which searches for the parameters that minimise the loss function.

Gradient descent (GD) is shown in Equation 18. The gradient descent updates the parameters for the whole dataset one time per epoch. The drawback of GD is that many times the learning rate is high and this makes the solution of the optimisation problem difficult since it is less likely to converge to an optimal point because usually, the step size is too large. For this reason, Stochastic Gradient Descent (SGD) was introduced by Robbins [2007]. SGD takes a small set of samples from the training set called minibatch, and it updates the parameters after N_b data points.

$$\theta_i = \theta_{i-1} - r\frac{1}{N_b}\sum_{j=1}^{N_b}\frac{\partial C_j}{\partial \theta}\ , \tag{18}$$

where the gradient is evaluated at θ_{i-1}. The minibatch size is usually a small number in order to surpass the limitations of Gradient Descent that uses the whole training set to update the parameters. It can provide an unbiased and fast way which updates the parameters more frequently.

One crucial optimization algorithm is ADAM which was introduced by Kingma and Ba [2015]. While SGD takes into account only a single learning rate for all its parameters updating procedure, ADAM utilizes the benefits of AdaGrad and RMSProp, and it uses per-parameter learning rates update [Ruder, 2016]. To be more specific, ADAM estimates an exponential moving average of the gradients (1st-moment estimate) and an exponential decay average of the squared gradient(2nd-moment estimate). The mathematical expressions of the first and second moment estimate respectively are:

$$M_i = \beta_1 M_{i-1} + (1-\beta_1)\nabla L_t(W_{i-1})\ , \tag{19}$$

$$R_i = \beta_2 R_{i-1} + (1-\beta_2)\nabla L_t(W_{i-1})^2\ , \tag{20}$$

where β_1 and β_2 are the the decay rate of the 1st and 2nd moments respectively. Weights update is proportional to the next ratio:

$$\frac{\text{Average gradient}}{\sqrt{\text{Average squared gradient}}}\ . \tag{21}$$

So the weights update for every epoch can be described by the next equation:

$$W_i = W_{i-1} - r\frac{\hat{M}_i}{\sqrt{\hat{R}_i + \epsilon}}\ , \tag{22}$$

where $\hat{M}_i$ is the first-moment bias correction, $\hat{R}_i$ is the second-moment bias correction,L is the loss function, and ϵ is a smoothing term. ADAM is computationally efficient cause it requires little memory resources.

2.3.6 Synopsis and softmax classification

To summarize, what most people call neural network is the Multiple Layer Perceptron. The most common form of MLP is the feed-forward MLP that is examined before, and a special case of it is the Convolutional Neural Network. A neural network consists of multiple hidden layers. Every hidden layer consists of neurons; the output of each neuron is passed through a non-linear activation function to the next hidden layer. The output of the last hidden layer is passed through either a sigmoid function when it comes to binary classification or a softmax function when we deal with a multiclass classification problem in order to provide an outcome which is transformed into probabilities. It is worth mentioning that there a lot of hyperparameters in Neural Networks. Some of them are the number of layers, the nodes on each layer, the activation functions and the learning rate.

Softmax is a common function in deep learning, and it is also known as multinomial logistic regression. Softmax normalises the network's predictions and convert them into probabilities which have a sum equivalent to the unit(1). Cross-entropy is utilised by the softmax function in the sense that the latter converts the outputs of the last layer into a probability distribution. Cross-entropy is used to measure the discrepancy between the output distribution, that a softmax generates, and the original distribution of the actual classes. Next, backpropagation is implemented in order to fix this error. Softmax can be described for all the outputs of a layer as:

$$\alpha_i^L = \sigma(z^L)_i = \frac{e^{z_i^L}}{\sum_{i=1}^{k} e^{z_i^L}} \ , \tag{23}$$

where k is the number of classes.

2.4 Convolutional neural networks and Transfer learning

Under the context of trying to explain how the human brain processes visual, auditory or other signal, scientists developed different algorithms for this scope. A deep learning approach that gained popularity, besides the plenty variants that are generated through the last couple of years, is Convolutional Neural Networks. A digital image is a 2D grid of pixels which has height, width and channels. Convolutional layers which are at the first part of the network learn low-level features such as lines and edges while the high-level features are built on top of the first layers, and they are used to detect objects and larger shapes.

The structure of a ConvNet is composed of three components. The first stage consists of two categories of layers: convolutional layers and pooling layers. Essentially, the convolutional layer is described by the convolution operation. The output of the convolutional layer is passed through a non-linear function to the pooling layer.

2.4.1 Understanding the convolution

Each unit of a convolutional layer is connected with units of previous layers through local patches of a generated feature map, and the connection is accomplished using filters or kernels. Filters are essentially the weights of the network. A visual representation of how convolution operation work can be shown in Figure 3.

So the convolutional layer receives an image and starts sliding the filter across the input image computing the element-wise product between the filter and different patches of the image. After the filter slides the whole image, a feature map is generated. The feature map indicates when the filter activates specific parts of the image, and those activations represent some general features like edges, corners or colours at the first layers or more detailed features at the last layers of the network. It is interesting that the features of the deeper layers usually cannot be interpreted by humans.

One analogy with the mechanism of MLP we have seen before could be that the values of the filters can be considered as the weights that connect the input of a typical neural network with the next layer before the activation function is applied. So, while an MLP uses matrix multiplication, a ConvNet uses the convolution operation. The only difference is that under the context of CNN's the filter remains the same for the entire image, while the weights of an artificial neural network are different for every pixel of the input image. This is called weight sharing because different units can use the same weights. Weight sharing has an impact in the sense that it reduces the number of free parameters in the model.

The aim of the described procedure is to collect together a few convolutional layers and to let the network learn different patterns and information, taking into account the spatial dependencies in the image.

Understanding the mechanism of a ConvNet is of high importance for this project. So it is crucial to introduce some terminology. First, the *receptive field* is the size of the filter (kernel). Filters are usually smaller than the input image because their utility is to focus on specific areas of an image and extract insightful features that occupy only a small proportion of the whole image. For example, a

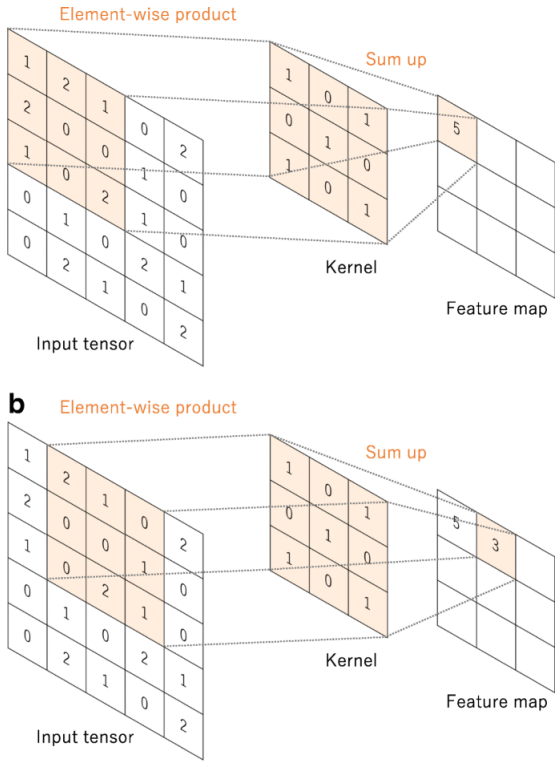


Figure 3: An illustration of convolutional operation [Yamashita et al., 2018].

filter of 4×4 informs that the height and the weight is equivalent to four. A neuron of a convolutional layer which has $4 \times 4 \times 3$ dimensions is essentially an RGB image with a total number of parameters equivalent to $4 \times 4 \times 3 = 48$ plus one for the bias.

2.4.2 Padding and Striding

After the convolution, the output is determined by some hyperparameters, such as:

1. **Depth**: The depth of the output is determined by the number of filters that are used for the convolution operation on the input image. Essentially, the depth column indicates how many times the set of neurons of the same area of the image were examined in order to reveal specific features.

2. **Stride**: The stride is the way the filter is sliding across the image. So when the stride is equivalent to the unit(1), then the kernel is moving across the image by one-step movements.

3. **Zero-padding**: Zero-padding can be described as the procedure of filling the input image with zero's around the limit of it. The following rule indicates how many neurons are generated as output when the convolution operation is implemented.

$$(I - R + 2P)/S + 1$$

, where I is the input size, R is the receptive field, P is the padding and S is the striding.

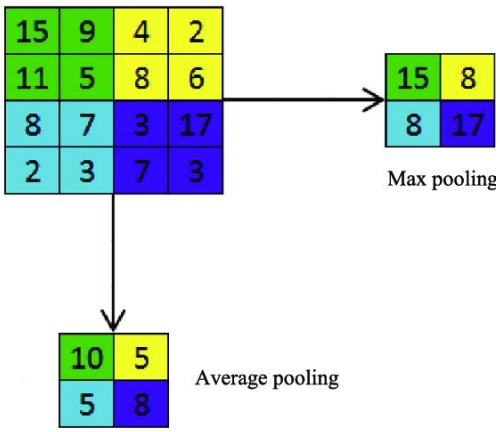


Figure 4: An illustration of max pooling and average pooling with 2×2 windows [Rawat and Wang, 2017].

2.4.3 Pooling

After, the convolution layer output many feature maps. Non-linear functions are implemented element-wise to the generated feature maps. ReLU is not only used under the MLP framework, but it is also a widely-used threshold for ConvNet's. Next, the *pooling layer* follows. Pooling is used to down-sample the feature map in such a way that the remaining features after pooling are the ones that are the most important when it comes to getting the most insightful information from the network. To underpin the previous argument, the features that remain after pooling are not easily affected to small movements on the input images, so the network can generalize better when some modifications implemented to the input images such as cropping or rotation. Pooling helps the network because it makes the features more stable (invariant) to small translations of the input.

Two pooling strategies have been mainly adopted by the researchers. The first is the*max pooling* while the second is the *average pooling*. Both techniques work like a filter which slides at the feature map similarly to the way that the filter slides across the input image at the convolution operation. Every region is represented by a different colour and max pooling, as the name implies, creates a matrix by keeping only the maximum value of the different regions, as we can see in Figure 4. Hence, max-pooling is implemented on the feature map we got after the convolutional operation, and a downsampled version of the feature map is returned as the output after max-pooling. On the other hand, on average pooling, the average value of the pooling window is calculated.

Finally, the fully connected layer is assigned to the already constructed architecture in order to transform the previous features into global features and to estimate the output score of each class. These layers expect one-dimensional vectors instead of the output of the convolutional and pooling layers, which are three-dimensional vectors. For this reason, the output of the last pooling layer is flattened in order to be transformed into a one-dimensional vector (what the fully-connected layers expect). Weights and biases are updated using the backpropagation algorithm, and the procedure is identical to the shallow networks we describe in section 2.3 [Lecun et al., 2015]. Ultimately, all ConvNet neurons of the layer before the first fully-connected layer represent the high-level features and are not easily explainable by humans. The fully-connected layers aim to map these high-level features to the output and generate the predicted classes of the network.

2.4.4 ConvNets architectures and VGG16

VGG16 took the second position in ImageNet Large Scale Visual Recognition Challenge (ILSVRC) while the first award was given to the GoogLeNet architecture [Russakovsky et al.]. As it was mentioned in section 2.4, convolutional neural networks consist of three core parts. The first is the convolutional

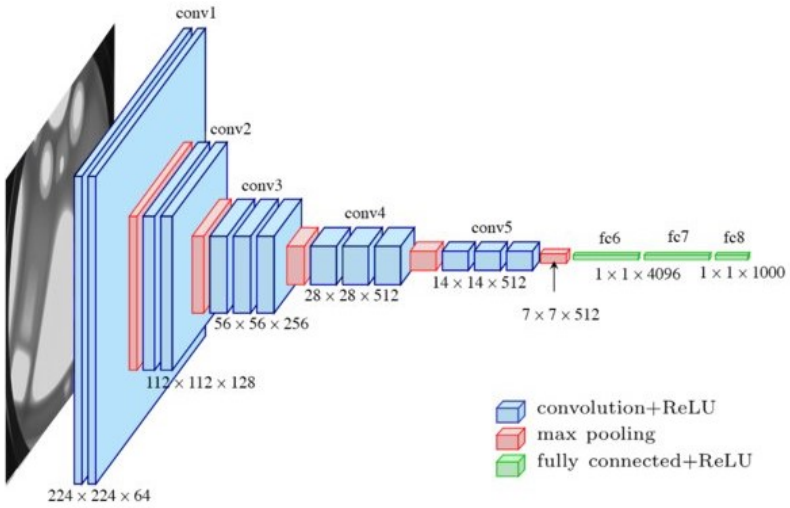


Figure 5: An illustration of VGG16 architecture proposed by Simonyan and Zisserman [2015] and visualised by Ferguson et al. [2017] in the referenced paper.

layer, the second part is the pooling layer, and the fully-connected layer is placed at the end. VGG16 is a state-of-the-art deep neural network that was proposed by the Visual Geometry School of Oxford University. It is widely used, and one of its benefits is that it does not require high memory resources compared to the VGG19 architecture. As a preprocessing step, the model expects a fixed size of 224×224 RGB images. The structure of this model consists of 16 layers. First, the images are given to the network and are passed through 5 blocks of convolutional layers and max-pooling layers. The max-pooling layers are represented with the red colour, as Figure 5 indicates, are used to downsample the output of the convolutional layers and are placed next to them. So the output of each convolutional layer is passed to the max-pooling layers, and spatial pooling is conducted for each one of the five convolutional blocks. The size of the max-pooling window is 2×2, the size of the filters are 3×3, and the stride is equivalent to 1. In addition, three Fully-Connected (FC) layers follow the block of the five convolutional layers. FC layers are responsible for dealing with the classification task. The first and the second FC layers have 4096 channels while the third and final FC layer has 1000 channels as the proposed architecture was engineered to solve a 1000-classes classification problem. Finally, the soft-max layer is added to convert the output into probabilities. The non-linear ReLU activation function was used throughout the whole architecture [Krizhevsky et al., 2012].

2.4.5 Efficiency through transfer learning.

Convolutional neural networks(CNN) gained prominence because they can achieve great results when they receive many training data as input while their performance is relatively low when there is a shortage of available data. According to Csurka et al. [2004] the first released CNN models had a rich performance even when compared with the Neocognitron, which was proposed by Fukushima [1980] and is considered the ancestor of the CNN. Despite the limitations of CNN's, Krizhevsky et al. [2012] model architecture lays the foundations of the new era for CNN because they accomplished to win the ImageNet Large Scale Image Recognition Challenge. ImageNet is a famous dataset which consists of approximately fifteen(15) million images and twenty-two thousand(22,000) classes. To continue with, Szegedy et al. [2015] came up with a new architecture which called Inception, and they achieved to beat Krizhevsky's model performance. The VGG16 network, which is of primary interest in this work, was introduced by Simonyan and Zisserman [2015] who were members of the Oxford Vision Geometry Group.

Even though these models accomplished very accurate results, they have a twofold limitation on the applications. The first limitation is that the computational power required to train the models from scratch is exhaustive, even for the most state-of-the-art computers. The reasons that lead to the enormous computational requirements are the complexity of CNN's and the high number of parameters which are trainable with the training dataset. The second limitation is the shortage of available datasets in domains such as medical imaging which can lead to poor predictive performance because the models could not sufficiently be trained.

Essentially, what is suggested in this work called transfer learning, and it can be considered as the solution to the above limitations. Transfer learning is described as the procedure of inheriting the prior knowledge from the source-task, which is obtained through the pre-training of the model with a publicly available dataset such as the ImageNet, to the target task problem we want to solve. Hence, transfer learning can assist the model with initial weights, and this can help the model to obtain a solution close to the optimal quickly. Having said that, it seems a reasonable choice to incorporate the transfer learning approach into our proposed model.

There is a lot of literature about transfer learning implemented on medical imaging. According to Xie and Richmond [2019], grayscale medical images need to pre-processed in order to fit with the colour images (3-channels) structure that used to pre-train the network. This can be accomplished with stacking the one channel (grayscale) images to 3-channel images and ultimately create a dataset with two more columns (second and third channel) equivalent with the first column (first channel). ImageNet colour images transformed into a greyscale image using the luma transformation technique. Essentially, luma is considered as the weighted sum of gamma RGB components of a colour image [Xiong and Pulli, 2010]. The contribution of this approach is that the ConvNet, which is trained using the grayscale version of the ImageNet, is more accustomed to dealing with greyscale inputs, and thus act as a better base for transfer learning. The filters that are implemented to the first layers are concentrated on the encapsulation of more general patterns, low-level features and colours. So training the ConvNet with greyscale images can assure that the results would represent better the greyscale images that are under examination. This research highlights the fact that pre-training a model with greyscale images can improve the classification performance both in accuracy and computational speed terms.

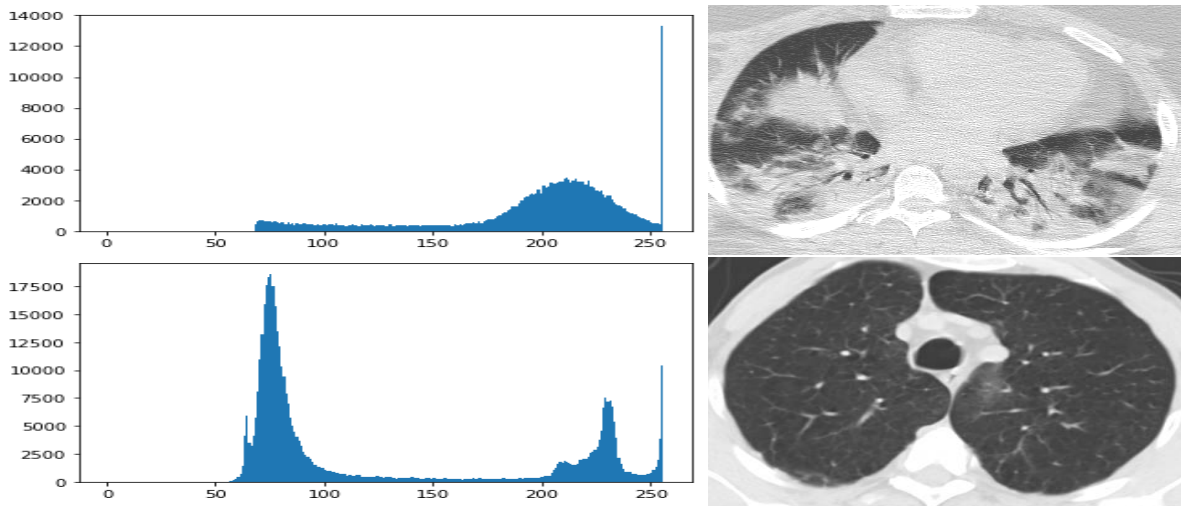


Figure 6: Top left and top right: Histogram of an image which represents a COVID-19 patient and the raw image of the aforementioned patient. Down left and down right:Histogram of an image which represents a Non-COVID-19 patient and the raw image of the aforementioned patient.

3 Data

3.1 Data sources and pre-processing

For this study, we analyzed a dataset which was retrieved from Kaggle. The data was gathered from hospitals in Sao Paulo, Brazil. The SARS-COV-2 Ct-Scan dataset consists of 1252 CT scans from patients that have SARS-CoV-2 virus and 1230 CT scans of non-infected patients. 32 males and 28 females were infected from SARS-CoV-2, while 30 males and 30 females suffered from a different pulmonary disease (non SARS-Cov-2) with similar symptoms. This dataset is almost perfectly balanced, and the images are greyscaled. There are no missing images, and the data collection procedure is described more thoroughly at Eduardo Soares et al. [2020] work.

Exploring the SarS-CoV-2 dataset helped us to understand what the different pixel values of the images represent and enable us to gain an intuition of our dataset. Hence, two histograms of a COVID and a non-COVID sample are presented. The horizontal axis represents the pixel values, and the range of the pixel values is [0,255]. 0 represents the black colour, while 255 represents the white colour. The values between those two extremes represent mid-tone values. Generally, the brighter an image, the closer to 255. Next, we visualize at the top part in Figure 6 one example of a COVID image and at the down part one example of a non-COVID image. It can be seen that the histogram of the COVID patient at the upper part of Figure 6 is more left-skewed than the histogram of the non-COVID patient, which is depicted at the bottom part of Figure 6. It is demonstrated that the COVID image is a brighter image than the non-COVID one and this can be proven by the fact that COVID samples have their most frequent values at the right part of the horizontal axis (left-skewed). On the other hand, we can infer that the non-COVID image at the down and right part of Figure 6 concentrates the majority of its pixel values on the left part of the horizontal axis, and as it is observed the non-COVID samples have a darker colour than the COVID ones. This insight would be taken into account in section 5 in order to draw an inference for what kind of images (COVID or non-COVID) are included in the different clusters.

The second dataset that is examined in this work called ImageNet and could be considered as the catalyst for the explosion of Artificial Intelligence that occured the last decade. ImageNet consists of 14,197,122 images and 22,000 classes. This huge dataset can be retrieved from the web and includes images of animals, breeds and daily objects. We use this dataset to pretrain our proposed models, as

it is described in section 4.

3.1.1 Data pre-processing and scaling

The COVID images have a different shape, and for this reason, they were scaled in order to have an equal size. The data were centred based on Imagenet mean. The mean value for the three channels (Red, Green, Blue) is subtracted from every pixel of the COVID ct-scans images. This is the default preprocessing for VGG16 model which has been accomplished using a method from Keras library which is responsible for the preprocessing of the input image.

To summarise, the dataset and the background have been described. So the theoretical background of this study has been set. The next two sections show how these foundations can be utilized to present a proposed methodology. The scope of section 4 and 5 is twofold. First to explore how modifications in the layer structure of the VGG16 architecture could affect the predictive results but also to provide an insight to the reader about how the selection of the right features could provide guidance for a more explainable model.

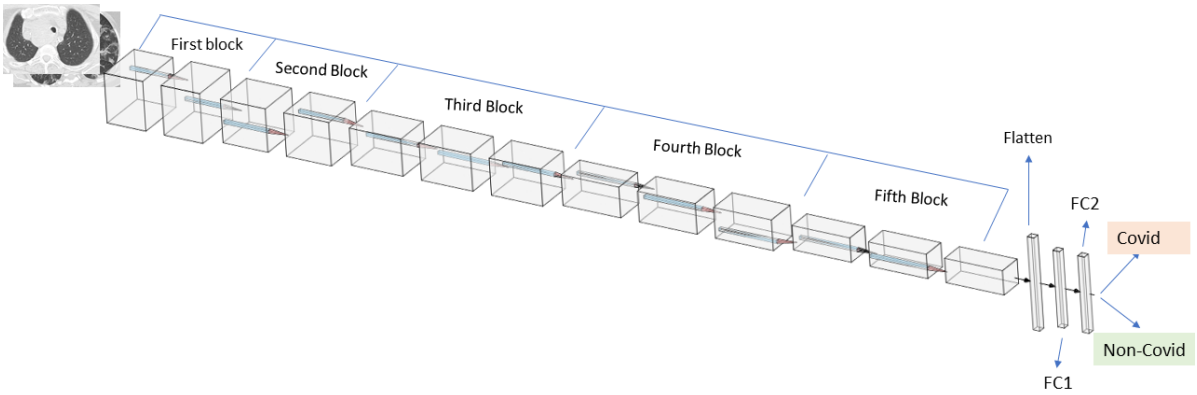


Figure 7: An illustration of our proposed architecture.

4 Classification using CNN

In this section, we present the modelling procedure, using the background methods we described in section 2. Also, the results of our proposed models will be compared.

The examined task is an image classification task. The features and the classes of the images were provided to us. Essentially, our scope is to implement supervised techniques in order to predict the target classes of the new features that our model have never seen before. Hence, predicting the labels of the test dataset is a cornerstone of our analysis and can provide great insights to domain practitioners while supporting them in their decision making process.

Traditional Neural Networks(NN) have a limitation that makes us not to consider them in our modelling procedure. This limitation is that a NN has a lot more parameters compared to CNN because the nodes of one layer are connected with all the nodes of the next layer, and this makes the whole procedure inefficient. On the other hand, as discussed in section 2.5, convolutional neural networks are more efficient at handling complex datasets, and they can locally summarise images using filters.

For this reason, ConvNets were used in this work. Taking into consideration that our dataset consists of approximately two thousand images, training the whole CNN from scratch does not seem a wise choice for two reasons. First, training a model from scratch is computationally expensive. Second, training a model from scratch does not ensure good results because it is usually challenging to estimate a model with such a small dataset robustly. For this reason, we used the VGG16 architecture, which was pre-trained on Imagenet dataset. We tried to develop a model which can yield superior prediction performance, while can ensure that the extracted features from the last layers are representative.

We first design an architecture that was constructed in such a way to match the class output. That is, the final fully-connected (FC) layer of the network is replaced by a new FC randomly initialized to match the number of output categories accordingly [Oquab et al., 2014]. A second way to handle the presenting challenges is by adopting a fine-tuning approach which can enhance the representativeness of the model on the target-task (COVID-19 classification) we want to solve.

4.1 Description of the proposed model

VGG16 architecture contains more than 138 million parameters. It is a very challenging task to train and optimise a model with 138 million parameters by using only 2000 images. Furthermore, the computational cost of trying to train the network from scratch makes this method inaccessible [Livni et al., 2014]. With these constraints in mind, the adopted strategy is to use the layers of the VGG16 model, which were trained with ImageNet data, as a feature extractor for low-level and mid-level

representation and then to transfer this knowledge to our dataset for high-level feature extraction. The described procedure is called transfer learning.

Transfer learning is more valuable when the available data of the source task is more than the data of the target task. In our case, we utilised the transfer learning by taking features learned on ImageNet dataset and used them for the COVID CT scans problem. The weights of the convolutional layers kept fixed to achieve quicker training. Additionally, transfer learning provides the advantage of non-random weight initialisation to the network.

According to Oquab et al. [2014], transfer learning has some limitations because the distribution and the classes of the images for the two different datasets (source and target) may be different. Our work is not an exception from these limitations as the two datasets, the ImageNet and the COVID-ct scans, are quite different. To deal with these limitations, we designed three different models which utilised the transfer learning capabilities, but at the same time, we constructed our models according to the requirements of the problem we want to solve (COVID-19 images classification).

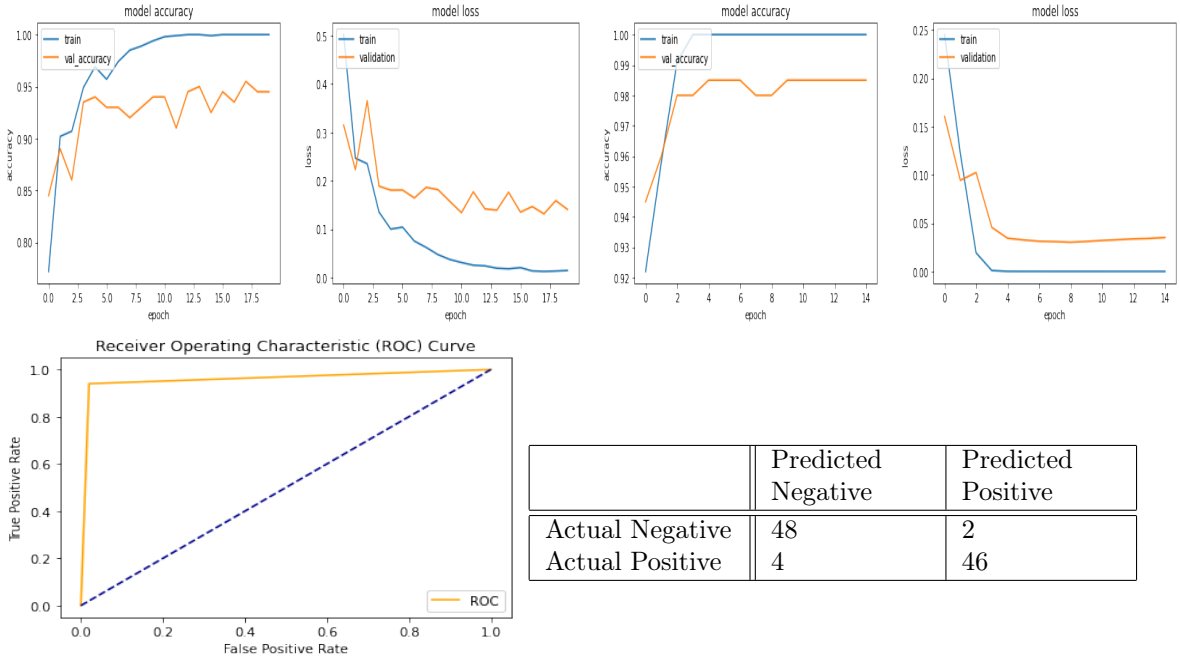
4.1.1 First method: Final Layer Transfer Learning with VGG16

The first strategy can be described as the most conservative one in the sense that we have not dramatically modified the parameters of the network. The first adopted method consists of a convolutional neural network which attains the architecture of VGG16 inspired by Simonyan and Zisserman [2015] as we can see in Figure 7. We pre-trained out model with the Imagenet dataset, and the only modification which conducted was to replace the third fully connected layer (1000 classes) with a new fully-connected layer (2 classes). That is, the last classification layer of VGG16 architecture was removed because it was engineered to output 1000 different classes and a new classification layer with an output of two (2) labels (COVID and non-COVID) replaced it in order to match the two-class classification problem we want to solve. This last fully connected layer receives as input, the output of the second dense layer, which contains a combination of mid-level and high-level representations of the training images. So we can say that $FC3 = \sigma(WFC2 + B)$, where W are the weights of the FC2 layer and B its biases. Next, all the layers apart from the last new one were frozen in order to retain the weights from the Imagenet dataset and only the last fully connected layer trained with our target-task dataset (COVID ct scans).

As an extension to transfer learning, it is crucial to introduce the fine-tuning. Fine-tuning is a way to optimise the parameters of the first layers which were trained on a different task (source-task) and then adjusts the weights from the target-task in order to produce better classification results. Adjusting the weights usually requires a lower learning rate than the initial learning rate used to train the CNN. By just slightly modifying the weights, we accomplished to reduce the computing power that may be required in the case that the weights were initialised randomly. Fine-tuning can either be implemented to all layers or to the high-level layers. The choice depends on the level of overfitting.[Nogueira et al., 2017]. In this work, we tried the first approach as we do not deal with a significant level of overfitting. Hence, fine-tuning was implemented as a complementary approach for every model we constructed, and it enhanced the performance of them.

Having said that, what we tried next is to unfreeze all the layers in order to fine-tune via back-propagation the weights and to make them adapt to our target images. We used a lower learning rate (0.00001) than the initial rate (0.1), and this technique brought an enhancement in a plethora of performance metrics such as accuracy, precision and AUC.

The training and validation accuracies and losses obtained by the model of the first method are presented in Figure 8. As we observe, the training and validation accuracies and losses were wavering before implementing fine-tuning (1st and 2nd subfigures) but they have been smoothed after fine-tuning (3rd and 4th subfigures). Those fluctuations were possibly caused due to the fact that the VGG16 is not a very deep model. After fine-tuning, we can see that both training and validation losses converged faster compared with the losses before fine-tuning. Also, a smaller number of epochs was used for fine-tuning because the models accomplished to converge quickly. The gap between training and validation accuracies and losses is not significant, so hard overfitting is not the case here. However, this issue



	Predicted Negative	Predicted Positive
Actual Negative	48	2
Actual Positive	4	46

Figure 8: Top-left: Training (blue) and validation (yellow) accuracy and loss for the first method (without fine-tuning). Top right: Training and validation accuracy and loss for the first method Down (with fine-tuning). Down left: Roc curve for using transfer learning to the whole architecture without fine-tuning Down left: Roc curve for the first method without fine-tuning. Downright: Confusion Matrix for the first method without fine-tuning. **(Method 1)**

could be solved by making the model larger, e.g., considering more training parameters into our model.

4.1.2 Second method: Five last Layers Transfer Learning with VGG16

The second approach can be described as less conservative because we kept only the feature extraction part of the pre-trained on ImageNet VGG16 model(e.g. until the fifth block of convolutions) and we added a new *set of layers*. That is, the new *set of layers* consists of one max-pooling layer which was used to downsample the output of the last convolutional layer from $14 \times 14 \times 512$ to $7 \times 7 \times 512$. Next, a flatten layer was added, and it results in a one-dimensional vector of size 1×25088. Afterwards, three fully-connected layers were attached. Essentially, this architecture is the same as VGG16 architecture which motivates us on the model building, but it uses different weight initialisation strategy because the knowledge is transferred from the last Convolutional block (Block 5) that trained with ImageNet dataset to the new *set of layers*. It is crucial to highlight that the new set of layers trained on the target task (COVID ct scans) while the weights of the previous layers were frozen. We can see a visual representation of the second proposed model in Figure 9.

To further improve the results, fine-tuning was implemented in order to make the model more compatible with the requirements of the target problem (COVID ct-scans classification). For this reason, the entire model was unfrozen and fine-tuned with a lower learning rate (0.00001). Higher learning rates were also implemented, but the model accomplished to converge successfully with only a very small learning rate like the mentioned one. This behaviour caused because higher learning rate usually forces the loss function to pass the global optimal point, and leads to high divergence. Hence, better results were accomplished with a lower learning rate, and the model achieved to generalise better on unseen data than our first model.

The training and validation accuracies and losses obtained by the second adopted method are

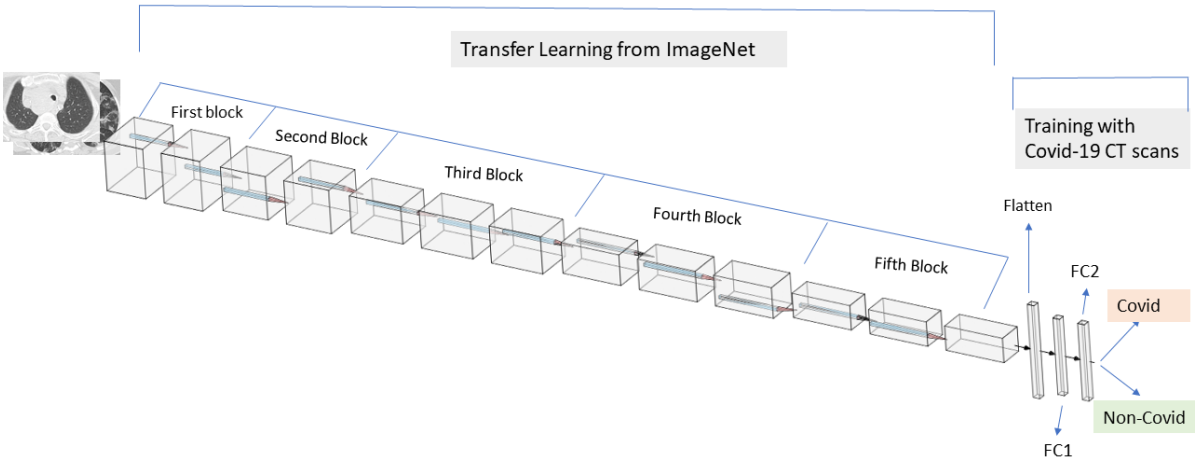


Figure 9: Our proposed architecture for the second method.

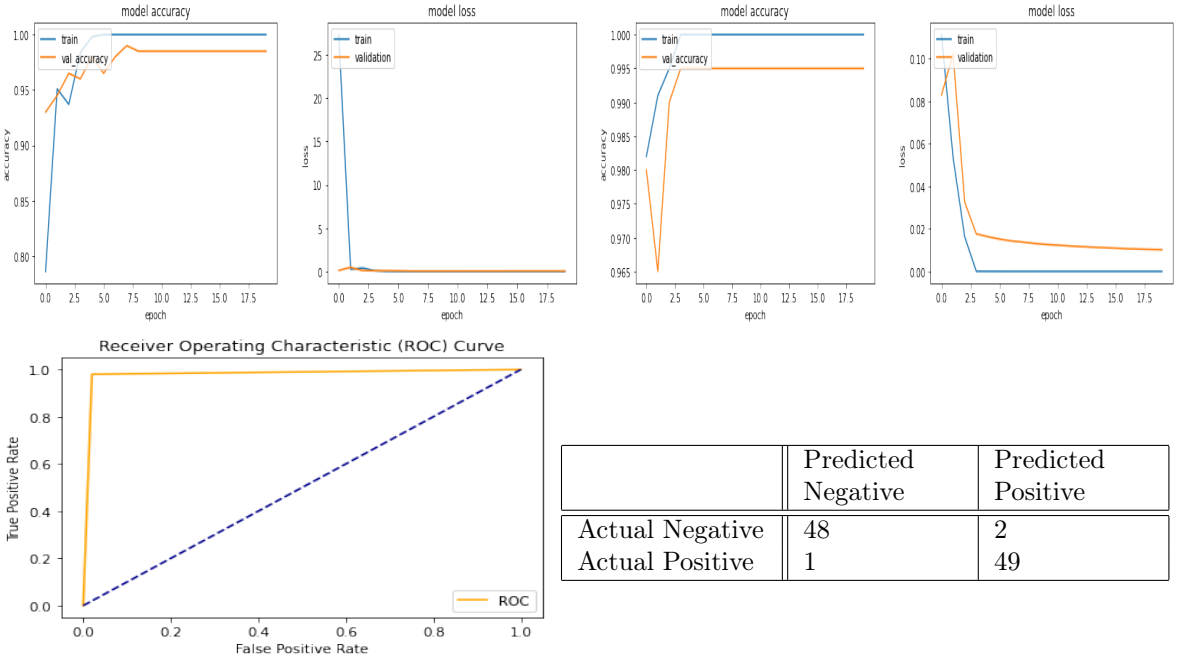
presented in Figure 10. As we have seen in Figure 8 there was a small gap between training and validation which may be an indicator of little overfitting. Our second adopted method aimed to alleviate this phenomenon by introducing more trainable parameters into the model. As we see in 2, four out of five evaluation metrics increased, while precision remained stable. Also, the loss function of the second model converged faster than the loss function of the previous model. This is consistent with the theory [Oquab et al., 2014] because transferring knowledge from earlier layers of a pre-trained network (method 2) in order to solve the target problem, appears to perform better. The reason behind this finding is that the earlier features of the source task contain more general patterns (e.g., edges, corners, etc.), and as a result, those early features can be proven more useful at transferring knowledge to a different dataset, such as the target one.

In our experiments for the two discussed methods, FC2 has a 4096 size while the last output layer has a size of 2, equivalent to the number of target dataset classes (COVID,non-COVID). The parameters of convolutional layers trained first on the source task then transferred to the target task and kept fixed. The first approach adopts the training technique of only the last dense layer which is responsible for classification while the second approach suggests to go deeper and to train the last five layers with the target images (COVID ct scans).

4.1.3 Third method: Mid-layers Transfer Learning with VGG16

Our third adopted strategy is influenced by the insight of Nogueira et al. [2017] who concluded that the earlier layers (low-level) of a network contain more abstract features of the training images, while the last layers contain more specific features of the images. Thus, when there are two different training datasets, there are higher chances to find more common features and patterns between those two at their first layers features (low-level features) rather than their last layers (high-level features). We use this information to understand that there are higher chances that the first layers of the VGG16 network, which is pre-trained on the Imagenet dataset, contain features which could be found at other images (like the COVID-ct scans) and they are not so specific. On the other hand, going deeper into the network, the features become more specific, and the transferred knowledge from the source-task to the target-task (COVID images) is not very representative.

With this in mind, we examine the case of transfer learning until a mid-layer of the VGG16 network, which is the fourth block of convolutional layers. According to Yosinski et al. [2014] when the target dataset is small, and the lower level filters cannot learn well when they are trained from scratch it is suggested to use the lower level features from the base network which was trained on a big dataset (e.g. Imagenet). In other words, we extract the features of the pre-trained VGG16 network on Imagenet



	Predicted Negative	Predicted Positive
Actual Negative	48	2
Actual Positive	1	49

Figure 10: Top-left: Training (blue) and validation (yellow) accuracy and loss for the second method (without fine-tuning). Top right: Training and validation accuracy and loss for the second method Down (with fine-tuning). Down left: Roc curve for using transfer learning to the whole architecture without fine-tuning Down left: ROC curve for transfer learning the model the whole model except the last five (5) layers (without fine-tuning). Downright: Confusion Matrix for the second method without fine-tuning. **(Method 2)**

dataset which reaches till the fourth block of convolutional layers (mid-level features) as it can be seen in Figure 11 and did not use the fifth block of convolutional layers of the VGG16 architecture. As discussed by Ng et al. [2015] the low-level layers of a ConvNet perform better than high-level layers because the features on the low-level and mid-level layers enclose more general elements of the images and due to the fact that our dataset (sars-cov-2) is quite different from the Imagenet dataset, we adopted this third strategy for training our model. Essentially, the features transferred from a mid-layer (Fourth Block) to the classification part (five last layers) and the training with our images was conducted only for the classification part of the network. In other words, the first part of the network was frozen. The classification part consists of a max-pooling layer which was flattened and passed through a non-linear relu function to the two fully-connected layers. Finally, a softmax with two classes was added because we deal with a binary classification problem. In this case, we utilised the transfer learning by transferring the knowledge of the more general features (shapes, curves), and we disregarded the high-level network weights derived from the ImageNet data. What followed next was the training of the classification part with our dataset.

The three adopted strategies do not reveal very big difference in performance. However, we consider them in order to understand how they behave under the context of clustering and to examine whether those difference in the modelling procedure can provide more explainable results. This is analyzed thoroughly in section 5. As we can see at the upper part of Figure 12, the results are again quite impressive.

The images that fed into the models retained a 224×224 pixels shape because this is the compatible input for the VGG16 architecture. State-of-the-art GPUs enhanced the performance and decreased the computational time needed as the system can process the data more efficiently. Although, according to

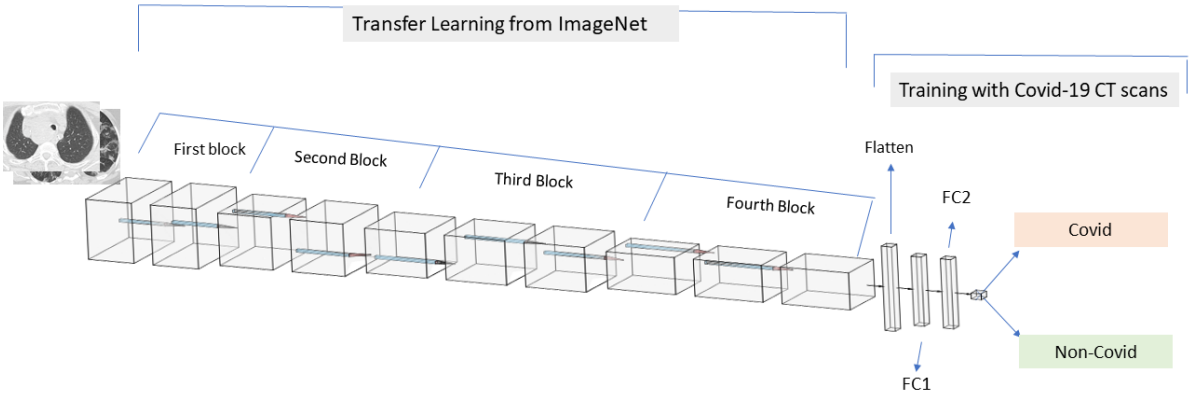
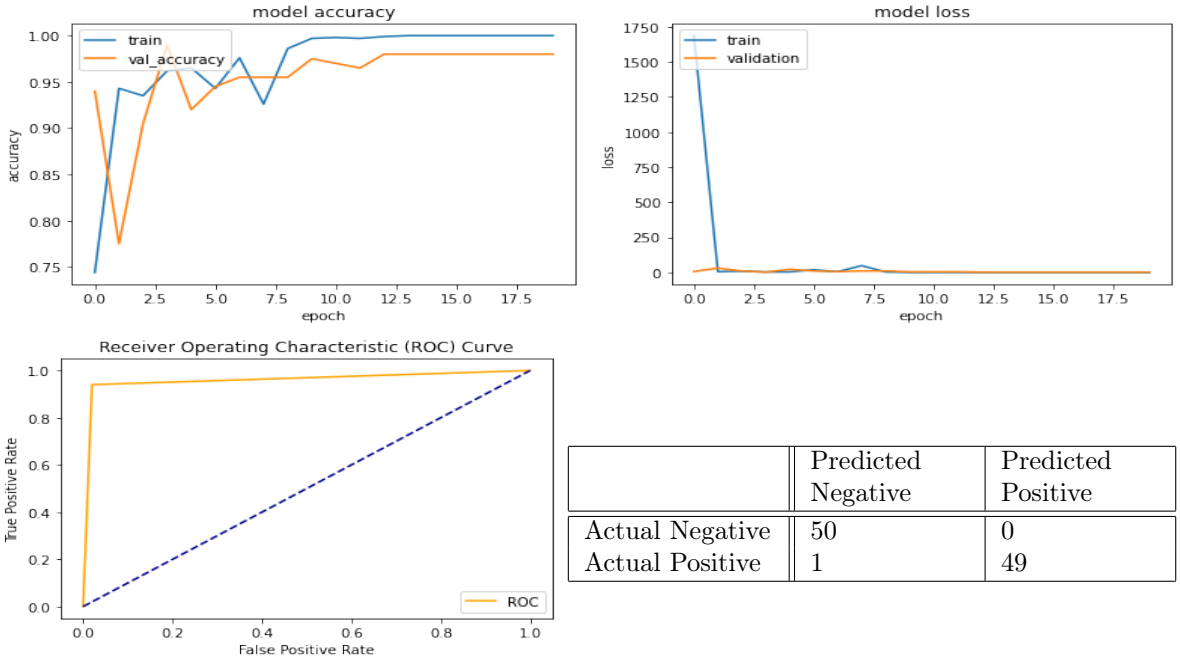


Figure 11: Our proposed architecture for the third method.



	Predicted Negative	Predicted Positive
Actual Negative	50	0
Actual Positive	1	49

Figure 12: Up left and up right: Training and validation accuracy and loss for the third method(without fine-tuning). Down left and down right: Roc curve and confusion matrix for method 3. **(Method 3)**

Masters and Luschi [2018] using a small number of batches (2 to 32) is more computationally-efficient, and has been shown to lead to better performance and lower training time. For this reason, we used batch size equal to ten and did not push to the limits the capabilities of the GPU. [Masters and Luschi, 2018]. The selected optimizer we used is Adam, and the chosen loss function was the categorical cross-entropy. The learning rate was set to 0.1 for the last layers trained with our images while the learning rate for fine-tuning our models was set to 0.00001. All the different strategies we tried converged after 15 epochs. Our experiments were implemented using a GPU (12.72 GB) from Google Colab platform. TensorFlow and Keras libraries were used as well in order to train and evaluate our models. Fine-tuning the models lasted for approximately 10 seconds per epoch.

4.2 Cross-validation and performance evaluation

To process with model evaluation, first, it is crucial to determine what the metrics reflect. A way to evaluate our algorithm is the cross-validation, which is a robust validation procedure because it repeats the procedure of training and validating the data k times (k is the number of folds someone chooses to use) and then averages the metrics, instead of just testing a single test run and produce one estimation. Specifically, I used one-Fold cross-validation, so the training data are divided into two subsets, and each time I used the one set for training and the other for validation. The ratio I picked to randomly split the dataset was 0.75 for the training set, 0.1 for validation and 0.15 for the testing set. We ensured that the validation images had not been used during the training procedure. This is of high importance as we managed to perform a classification task that produces valid and unbiased results.

For this analysis, the performance of the classifiers is assessed by measuring and comparing five different evaluation metrics (accuracy, precision, recall, F-score and AUC score). First, the confusion matrix for every classifier has to be constructed. Rows represent the true class while the columns represent the predicted class. When a class was predicted as positive, and its actual class is positive, then this is labelled as a True Positive (TP) while when a class is actually positive but is predicted by the classifier as negative, then it can be written as False Negative (FN). Similarly, one can construct a measure of True Negative (TN) and False Positives (FP). We can see all the possible combinations for the two-class classification (binary) problem in Table 1.

	Predicted Negative	Predicted Positive
Actual Negative	TN	FP
Actual Positive	FN	TP

Table 1: Confusion Matrix.

Accuracy is the ratio of the correct predictions that the model achieved to predict on the test dataset. Essentially, accuracy does not care about which of those are either positive or negative but only deals with predicting them correctly.

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \tag{24}$$

Precision focuses on measuring how many of the predicted labels of the class we are interested in are predicted correctly. So, precision estimates the confidence of the classification. Precision depends on the application of interest. For example, if the predicted COVID patients are 70 out of 100 and only 50 out of 70 were predicted correctly, we can say that precision = 50/70= 0.71.

$$PREC = \frac{TP}{TP + FP} \tag{25}$$

Recall, or sometimes called sensitivity, is a measure of how many examples of interest your model has identified as such. So, recall estimates how many of the actual labels of a class are finally predicted correctly. Supposing that we have one hundred (100) examples and we know that 70 of them are actually COVID, but our model only predicted as COVID 60 out of 70. Then the recall would be 60/70 = 0.86.

$$REC = \frac{TP}{TP + FN} \tag{26}$$

Usually, recall and precision are mutually exclusive. That is, when the precision is increased, the recall is decreased, and vice versa.

Performance metrics	Method 1	Method 2	Method 3
Accuracy	0.94	0.97	0.99
Precision	0.96	0.96	1
Recall	0.92	0.98	0.98
F1	0.94	0.97	0.989
AUC	0.94	0.97	0.99

Table 2: Performance metrics for the three (3) methods without fine-tuning

F-score is widely used when the dataset is not balanced and is the harmonic mean of recall and precision, while it takes values from zero to one.

$$F = \frac{2 \cdot PREC \cdot REC}{PREC + REC} \tag{27}$$

Receiver Operating Characteristic or (**ROC**) was employed in signal detection theory, and it was used as a tool for the analysis of noisy signals. Area under curve or **(AUC)** summarizes the trade-off between true positive rate (TPR) on the y-axis and false positive rate (FPR) on the x-axis [Trevor Hastie and Friedman, 2001].

$$TPR = \frac{TP}{TP + FN} \tag{28}$$

$$FPR = \frac{FP}{FP + TN} \tag{29}$$

Having said that, constructing a prediction with a classifier usually preassumes a decision boundary (threshold) in order to decide when a class can be classified as a positive or otherwise. A rule of thumb is to assume a threshold of 0.5. That is, when the predicted probability of one observation being positive is 0.8 (higher than the threshold of 0.5), then it is documented as positive. A good classifier would have a ROC curve which is as close as possible to the upper left corner (AUC=1) and as far as possible from the diagonal line (AUC=0.5).

To summarize, the different versions of the proposed models were trained, and their performance was evaluated. The suggested models fitted the different images and accomplished to produce some good results and to generalize well as measured on the test data (n=100). The results for the three methods can be summarized at the table (2).

However, the differences are relatively small; the results on Table 2 demonstrated that the third method accomplished to predict better the labels of the test dataset compared to the other two models. It is important to note that on medical sector, assessing the results only by comparing the accuracy metric is quite risky because there are high chances to overestimate the potential of the model by neglecting to focus on the cases that were incorrectly classified as non-COVID. Having said that, the cost of the cases that were predicted as non-COVID but they were actual cases of COVID (False Negatives) in medical sector is very high. The models with the highest Recall are the second and the third ones. However, the third model achieved better AUC and Accuracy results. Ultimately, the the main aim of the next chapter is to find out how the second and third models can be compared in terms of their clustering results.

5 Understanding Neural Network Activation

5.1 Feature extraction and Dimensionality reduction

There is work in the literature about using PCA to reduce the dimensions of the extracted features from a layer of a convolutional neural network and then passing them as input into a new classifier such as support vector machines [Razavian et al., 2014] or a multiple layer perceptron (MLP) to boost the prediction performance of the model [Bishop, 2006]. However, the scope of this section is not to expand more on the supervised learning side, as we succeed to catch some good results with our models (0.95-0.98 accuracy) but to try research how unsupervised learning can transform those images into something explainable.

Whilst deep learning models achieve some exceptional results, and the breakthrough they made to research is enormous, there is much criticism as they are considered a black-box procedure. For this reason, the next step of our analysis is an effort to understand the decisions taken by Neural Networks and to get insights rather than consider the whole procedure as a black box.

As we have seen in the previous section, one crucial property of the deep learning models is that they can be used to extract deep layer features. In this work, the features of the second fully-connected were extracted. These features were represented by a 4096-dimensional vector and contained useful information concerning the input images. To be more specific, the matrix which is produced from the second fully-connected layer of the CNN model is of size $F \in R^{n \times m}$, where $n = 598$ is the number of images and $m = 4096$ is the number of features that extracted. Specifically, due to computational restriction, we extracted the features of 598 images which are equally separated to 299 samples of COVID images and 299 non-COVID images. So, the generated matrix has a size of 598×4096.

State-of-the-art deep learning frameworks always incorporate computational methods which try to optimise the efficiency of their models. At the current stage, the dimensions of our dataset have reduced to 598×4096. So, we adopt a dimensionality reduction technique called principal component analysis (PCA) [Moon and Phillips, 2001] in order to further reduce the complexity of our dataset and as a result, to make it more efficient.

In Figure 13, we can see one visual example regarding the implementation of principal component analysis on one raw image of COVID ct-scans dataset. The left image depicts the raw image, while the right image depicts the first principal components that explain seventy per cent (70%) of the total variance of this specific image. As we can observe the second image is quite blurred, but the main patterns of the original have been preserved. Intuitively, we can say that the green colour area of the image in Figure 13 contains different information in comparison with the blue colour, which seems to be the outer part of the lungs. Domain knowledge is required to identify patterns further and critically characterise those images.

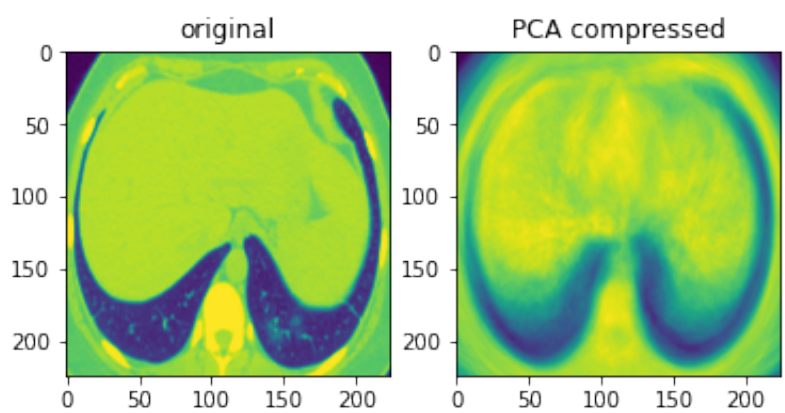


Figure 13: Left:raw image Right:First components that cover 70 % of the total variance.

PCA's objective is to find correlations between the features, to keep the most important which cover a sufficient proportion of the explained variance and to reduce the dimensions of our extracted dataset. Principal components indicate key directions in the data, and it can be implemented by eigenvalue decomposition of the data covariance matrix. By projecting the data from the z pixels onto k-principal components, we can achieve massive compression of the image data. PCA in the light of this specific work is used to project the 598 images hierarchically onto the most crucial eigenvectors and to produce a space (eigenspace) which consists of 598 points, each of them represents the principal components of each image. One adopted heuristic is to keep the principal components that can explain the ninety per cent (90%) of the total variance. The first principal component explains a higher proportion of the total information, which is included in each dataset and as we proceed to the next components, the noise increases. Generally, when the features(m) are more than the samples (n) then we need only n number of components to represent the whole original dataset, as when m>n the covariance matrix used to derive the principle components is only of rank n.

As a preprocessing technique, normalization is suggested in order to achieve results robust to changes in marginal variance and to ensure that noise is separated from the signal information. The results of the principal component analysis in this work will be examined below. The features from the second fully connected layer, for each one of the three adopted methods, was extracted. PCA is implemented in all the three FC2 extracted features aiming to reduce their dimensions further.

For the first method, as we can see at the top-left corner in Figure 14, we need 102 principal components to explain the ninety (90%) of the total variance cumulatively. This is a nice first improvement in comparison with the matrix with (598×4096) dimensions that were extracted. The amount of variance that the first five components accomplished to explain can be depicted in the top-right corner of Figure 14.

When the principal component analysis was implemented to the features extracted by the second method the results were different in the sense that the cumulative ninety per cent of the variance was explained by only the thirty (30) first principal components, as we see at the middle and left side in the Figure 14. As a result, the first principal component for this approach accomplished to explain a higher proportion of variance compared with the five (5) first components of the previous approach. This can be visualised in the middle and right part in Figure 14.

Finally, PCA was implemented to the features extracted following the third method, and the results were impressive as we succeed to reduce the dimensionality from a matrix of (598×4096) dimensions to a matrix of (598×5) dimensions. That is, as we can see at the bottom left corner of the Figure 14 the cumulative proportion of variance which was explained reaches the ninety (90%) taking into account only the first (5) components. The amount of variance that is explained by those five principal components is depicted by the graph on the bottom right corner of Figure 14.

5.2 Clustering using Gaussian Mixture Models

Clustering tries to find some homogenous subgroups among the observations, and can be considered as the microscope of the data science toolkit that enables the scientists to focus more on specific sides of the dataset.

The most classical tool for clustering is K-means. K-means is a hard-clustering method which means that it associates each point to one and only cluster. The limitation of this method is that there is no probability that indicates how much a data point is associated with the cluster. The Euclidean distance does not provide the flexibility that a probabilistic clustering approach could give, and it cannot be considered a robust clustering tool when the data points do not have a spherical shape, and the clusters overlap. In this work we used Gaussian mixture models which considers both the mean and the covariance structure of the data, which is an advancement compared to the k-means algorithm, which only examines the mean.

Gaussian mixture models (GMM) are probabilistic models that are responsible for the generation of distinct clusters. It is implied that they can be used as an extension of the k-means technique in the sense that GMM do not only assign a data point to some cluster but also, every data point is assigned

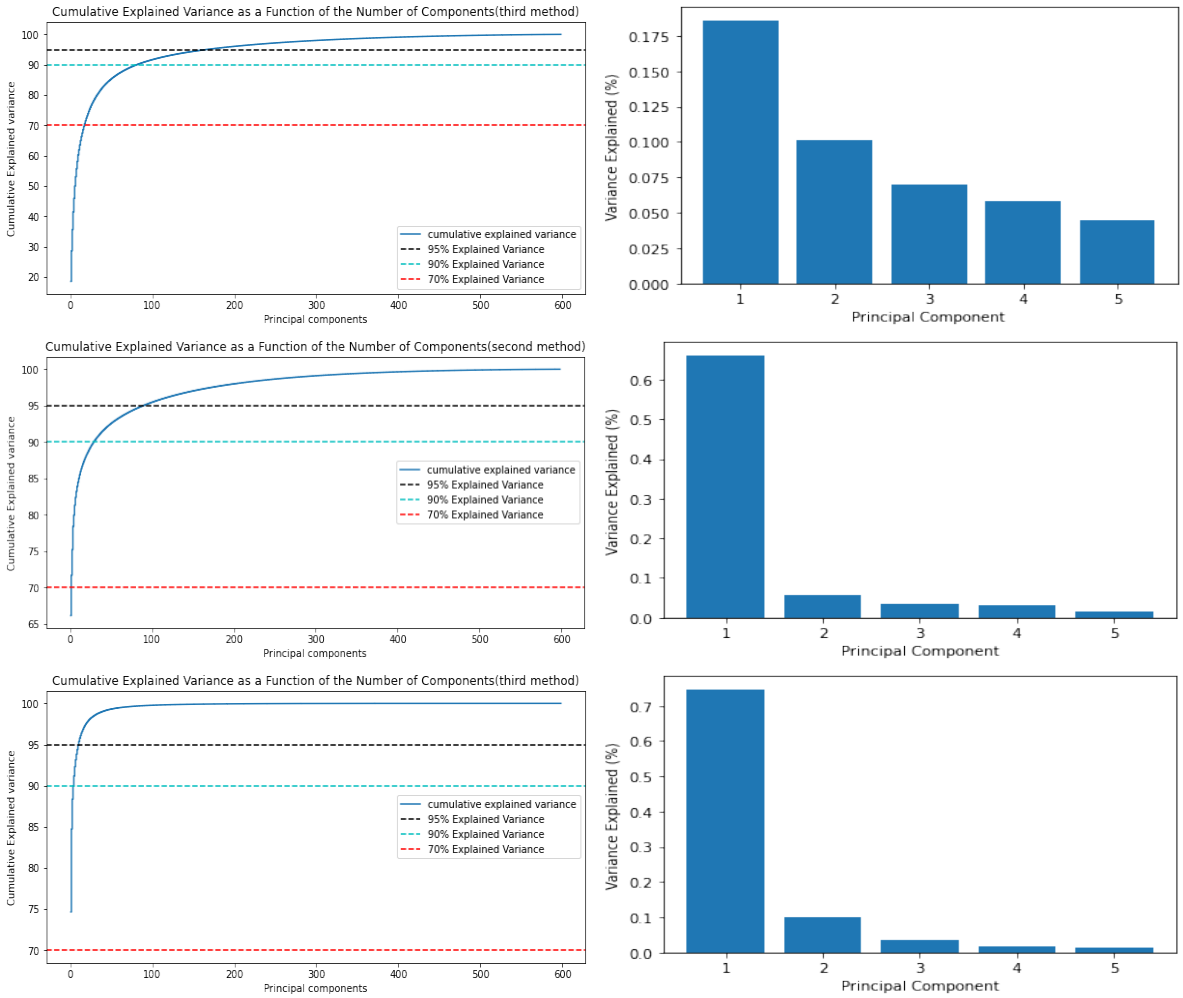


Figure 14: Top Left: Cumulative explained proportion of variance for the first method.Top Right: Proportion of variance explained by each one of the first five components for the first method. Middle left: Cumulative explained proportion of variance for the second method. Middle Right: Proportion of variance explained by each one of the first five components for the second method. Bottom left: Cumulative explained proportion of variance for the third method. Bottom Right: Proportion of variance explained by each one of the first five components for the third method.

a probability of belonging in a cluster. Hence, by using this technique, it is ensured that a data point is assigned a probability of being in each cluster, rather than strictly assigned to any individual one. Every cluster follows a gaussian distribution and has a mean μ_i and a variance Σ_i:

$$g(x;\theta) = \frac{1}{(2\pi)^{\frac{n}{2}}|\Sigma|^{\frac{1}{2}}} \exp\Big(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\Big) \tag{30}$$

For this reason, different cluster assignments are estimated over all the data points. A single cluster whose density is described by equation (30) has only one maximum point, but for different clusters, there are different peaks in the density. For a peak point, it is satisfied that:

$$(\mu,\Sigma)_{MLE} = \underset{\mu,\Sigma}{\operatorname{argmax}}\, g(x;\theta) \tag{31}$$

For this reason, the mixture of Gaussians contruct the overall density:

$$f(x;\theta) = \sum_{i=1}^{m} p_i g(x;\theta)\ , \tag{32}$$

where g is a Gaussian density with mean μ_i and variance Σ_i. Also, p_i is the mixing probability, e.g. the probability that the x observation comes from the ith cluster. The mixing probabilities have to sum to one (1) as equation (33) shows:

$$\sum_{i=1}^{m} p_i = 1 \tag{33}$$

We used the "Mclust" library to fit several Gaussian models to the data [Scrucca et al., 2016]. The mean and variance were calculated iteratively for each cluster. After the learning procedure, we estimated the probabilities that a data point belongs to each cluster. A usual pitfall is to get trapped in a local maximum point. Hence, we fit the GMM model with many initial guesses. Generally, the cluster assignments for all the observations were estimated, and the parametrisation was implemented by minimising the negative log-likelihood function. When the joint probability density function is considered as a function of θ for the x_j which are coming from the gaussian distributions g it is called likelihood function and it is denoted by $L(\theta|x_j)$. So, the likelihood function is the product of pdf for n data points:

$$L(\theta|x_1,x_2,..,x_n) = \prod_{j=1}^{n} f(x_i|\theta) = \prod_{j=1}^{n}\sum_{i=1}^{m} p_i g(x_j;\theta) \tag{34}$$

Next, the log-likelihood function :

$$l(\theta) = \log L(\theta|x_1,x_2,..,x_n) = \sum_{j=1}^{n} \log\Big\{\sum_{i=1}^{m} p_i g(x_j;\theta)\Big\}\ , \tag{35}$$

However, it is difficult for GMM's to maximise the log-likelihood directly via Maximum Likelihood procedure because we deal with a non-convex optimisation problem due to the component assignment. An iterative procedure called Expectation-Maximization (EM) algorithm is the proposed solution [Dempster et al., 1977].

Before getting into the details of our analysis, it is crucial to describe a technique which leads to the selection of the final model. The optimal number of clusters is selected using the penalised likelihood of training dataset(such as BIC). BIC can be expressed mathematically by the next equation:

$$BIC = -2\log(L) + m\log(n)\,, \tag{36}$$

where (m) is the number of parameters, (n) the number of observations and (L) is the measure of fit. The equation (36) consists of two parts. Generally, a model with a lower BIC value is better than a model with higher BIC value. The $(-2\log(L))$ part of the equation (36) rewards the model because it fits the data well. On the other hand, $m\log(n)$ increases as higher the number of parameters (m) and observations (n) becomes. It is implied that the larger the model is, the higher the penalty becomes. So, it penalizes for building a complicated model.

So far, the different images were classified, and the models predicted the classes of the test dataset. Next, the second fully-connected layer kept and its dimensionality was further reduced with Principal Component Analysis. The results of PCA produced a 2-dimensional matrix which is a low-dimensional representation of the raw images. This approach aims to suggest a novel method which analyzes, explains and supports the decisions made by the Convolutional Neural Networks(CNN's) models we discussed in section four (4). For this reason, model-based clustering, and especially the GMM, is further examined below.

The second suggested model required the first thirty (30) principal components to explain the ninety per cent (90%) of the total variance. Hence, a matrix of 598×30 was generated, and GMM was implemented. In extension to the K-Means algorithm, the Gaussian models are described not only by the mean but also by the variance-covariance structure, which describes the ellipsoidal shape. We select the optimal model by using the BIC criterion, and six clusters were suggested for the second method model while seven for the third method model. As we can see in Figure 15, the BIC criterium selected eight (6) clusters with EVE shape, BIC value equivalent to -143904 and log-likelihood equal to -69410.99. So the six (6) different clusters have equal volume and orientation but different shape.

As we can see at the right part of Figure 15, selecting 7 clusters also provide an equal BIC value. For this reason, the GMM components with seven clusters were experimentally implemented, and we inferred that the fourth cluster in Figure 15 could be represented as a mixture of two Gaussian components. BIC identified this information though, and the two clusters were concatenated into one. This experiment matches what Bouveyron et al. [2019] had shown at their work, and it can be summarized that two smaller clusters could be replaced by one bigger spherical cluster. The reason behind this replacement is that there is a trade-off between the number of clusters (model complexity) and the performance (BIC).

Next, we checked the cluster assignment of all the images. We considered that every image is assigned to the cluster for which it has the highest clustering assignment. This information motivates us to measure how many of the 598 images were assigned correctly. The results are quite promising as we can see in table 3. The GMM managed to assign 280 out of 299 COVID images to one of the four clusters (Cluster 2, Cluster 3, Cluster 4, Cluster 5) and 280 out of 299 non-COVID images were clustered to one of the two remaining clusters (Cluster 1, Cluster 6). The fact that the vast majority of the COVID images were more possible (higher cluster assignment) to be clustered to one of the four clusters (Cluster 2, Cluster 3, Cluster 4, Cluster 5) motives us to characterise those clusters as "COVID clusters". In contrast, the remaining clusters (Clusters 1 and Cluster 6) could be described as "non-COVID clusters" for the same reason. At the left part of the Figure 15 we can observe that the two non-COVID clusters are depicted by the blue colour points and are quite separated compared to the other four COVID clusters which are concentrated at the right part of the image.

As we have observed at equation (31) every cluster contains a point which is the maximum likelihood estimate for the different Gaussian distributions that represent the different clusters. Prototypes are highly representative images coming from the training dataset, and local peaks of the data density characterize them. According to Jain et al. [1999], prototype selection could be beneficial because it provides helpful insights about simple and stable patterns inside a dataset. This is called data abstraction and is a common non-parametric approach under the clustering context. Therefore, the

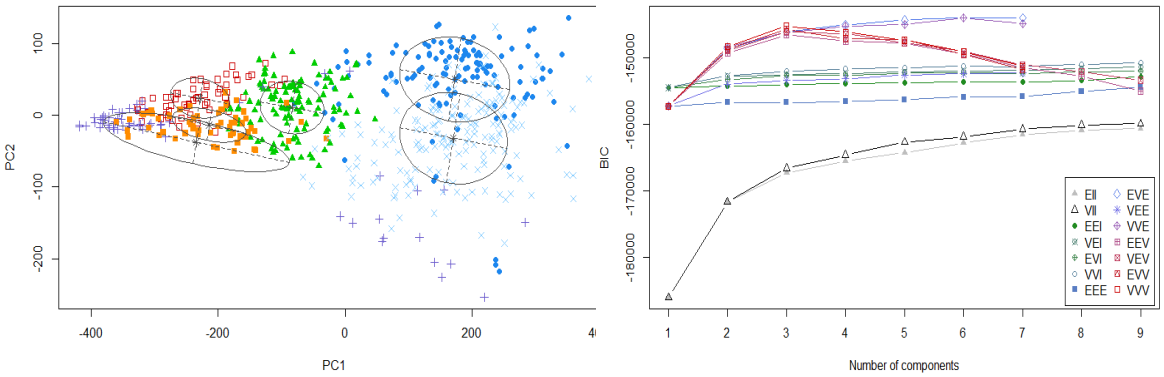


Figure 15: Left:GMM clustering for the first 30 principal components that kept after the implementation of PCA on the extracted features from the FC2. Right: Model selection via BIC criterium.**(Method 2)**

	Clustered as COVID	Clustered as Non-COVID
Actual COVID	280	19
Actual Non-COVID	19	280

Table 3: Evaluating the results of GMM clustering of our second model.

closest point to the cluster centre was retrieved by calculating the euclidean distance between every data point with each cluster mean [G. et al., 2005].

For our second method, the six (6) prototypes were extracted, and we can see them in Figure 16. The prototypes images for the first and sixth cluster were non-COVID images while the prototypes for the rest four clusters were images that had a COVID label. The mixing probabilities for the first and sixth cluster (non-COVID clusters) summed up to a total percentage of 50.1 % while the mixing probabilities of the rest four clusters (COVID cluster) summed up to a total of 49.9%, as we can see in Table 4.

	Cluster1	Cluster2	Cluster3	Cluster4	Cluster5	Cluster6
Mixing probabilities	0.21	0.098	0.19	0.11	0.10	0.29
Clustering table	127	59	114	66	60	172

Table 4: Mixing probabilities and clustering table for model-based clustering for the reduced features of the second proposed model.

The results showed us evidence that the four out of six prototypes had 100% probability of being in one out of four clusters, while the other two prototypes had 99% probability of being in the remaining cluster. So, some of the most representative points of each cluster (prototypes) were clustered with

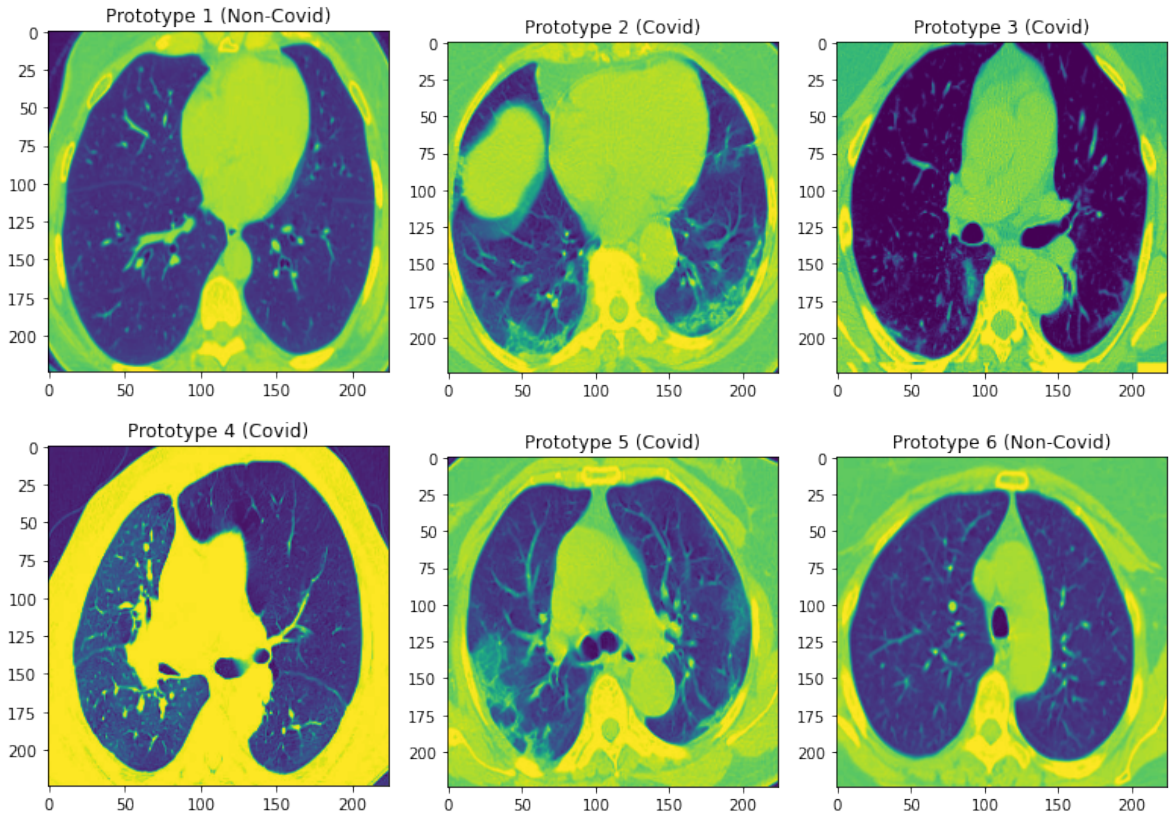


Figure 16: Prototypes images for the six clusters (Model 2).

little flexibility to belong in different clusters. This was the case for all of our images. Every image had six different probabilities, one for each cluster.

The next step in our methodology is to construct the average image for each cluster. This could be implemented by taking a weighted probability sum of the image values. For instance, we considered the ith pixel of the lth image x_{li}, and we weighted this according to the probability of assignment to cluster j:

$$I_{ji} = \frac{1}{n} \sum_{l=1}^{n} P(l \in C_j | x_{l1}, \dots x_{ln_{pixels}}) \times x_{li}$$

Next, we re-arranged the above back into an image, and we get a representative image for each cluster. For every image, we calculated the product of the conditional probability that the lth image owns to jth cluster $P(l \in C_j)$ and all the pixels for every image x_{li}. The average of this product generated the mean image. That is, we essentially constructed a mean image for every cluster which can give us an idea of what kind of images represent each cluster.

We can see the six different mean images in Figure 17. Intuitively, we can say that the results match what we got as insight by our previous analysis when we characterized the six different clusters, and we found what kind of images were encountered to each of them. As we have seen, the vast majority of the images in either cluster one (1) or cluster six (6) consisted of non-COVID patients, while the cluster two, the cluster three, the cluster four and the cluster five (2-5) enclosed COVID patients to a great extent. Cluster one (1) and cluster (6) mean images in Figure 17 can be distinguished from the other mean images in the sense that do not have a lot of white patches. The only exception may is the third cluster which looks like more a COVID one. At this point, the domain knowledge of a doctor would be helpful, and the fact that medical imaging stands at the intersection of data science and medical

science is confirmed. It is worthwhile to mention that the findings of our experiments are enforced by what we have seen at section 3, where the COVID images had more frequent pixel values closer to 0 (white) than 255 (black). That is, we got evidence that the white patches are probably indicators that someone is a COVID-19 patient. We can say that implementing GMM to the low-dimensional features extracted from the FC2 layer of our second proposed model achieved to generate six very representative clusters.

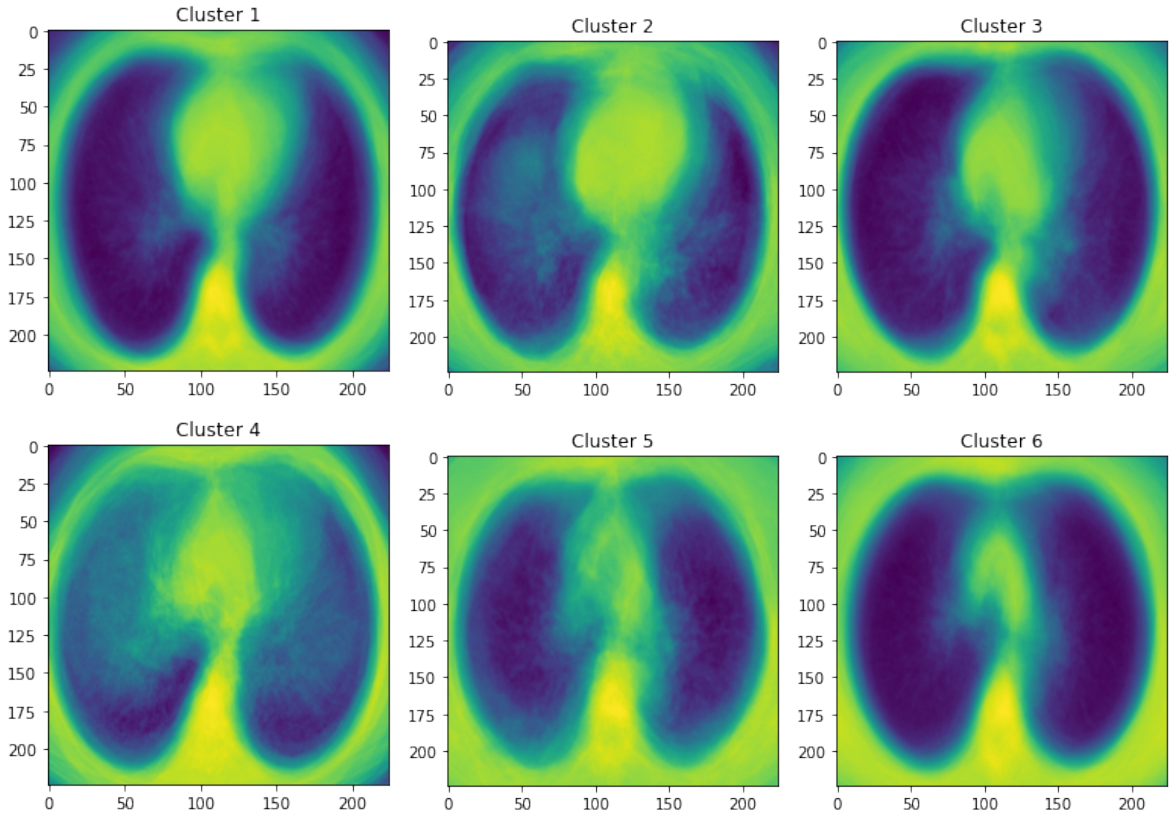


Figure 17: Mean image representation for the six clusters (Model 2).

The same procedure was replicated for the extracted features of the second fully-connected layer with our third proposed method. The principal component analysis accomplished to reduce the dimensions of the images to a 598×5 matrix. The BIC criterium selected seven (7) clusters with VVV shape. That is, the clusters have a different shape, volume and orientation. The BIC value was equivalent to $-51,010$ and the log-likelihood equal to $-25,038$. So, the log-likelihood of the third method is higher than the log-likelihood of the second method. Hence, the data fit better the third method's model. The reason behind this evidence could be that the model constructed with our third proposed method had been inherited with more general patterns of our source dataset and the trainable parameters for the part of the model which was trained with our target dataset (COVID ct-scans) were higher compared to the trainable parameters of the second model.

Again, the results of clustering were assessed by checking in which cluster the images were assigned. The results are quite promising as we can see in table 5. The GMM assigned all cases of non-COVID images to one out of the four non-COVID clusters (Cluster 4, Cluster 5, Cluster 6 and Cluster 7) while it assigned 281 out of 299 COVID images to one of the three COVID clusters (Cluster 1, Cluster 2, Cluster 3). At the left part of Figure 18, we can observe that the three COVID clusters positioned at the left side of the graph, while the four non-COVID clusters located at the right side. Also, the

orientation of the COVID and non-COVID clusters formed a perpendicular angle which is mainly because of the way that PCA creates the new orthogonal coordinate system. One reason may be that the first two principal components summarized the highest amount of total variance, so the information that first and second principal components enclosed is very different.

	Clustered as COVID	Clustered as Non-COVID
Actual COVID	281	18
Actual Non-COVID	0	299

Table 5: Evaluating the results of GMM clustering of our third model.

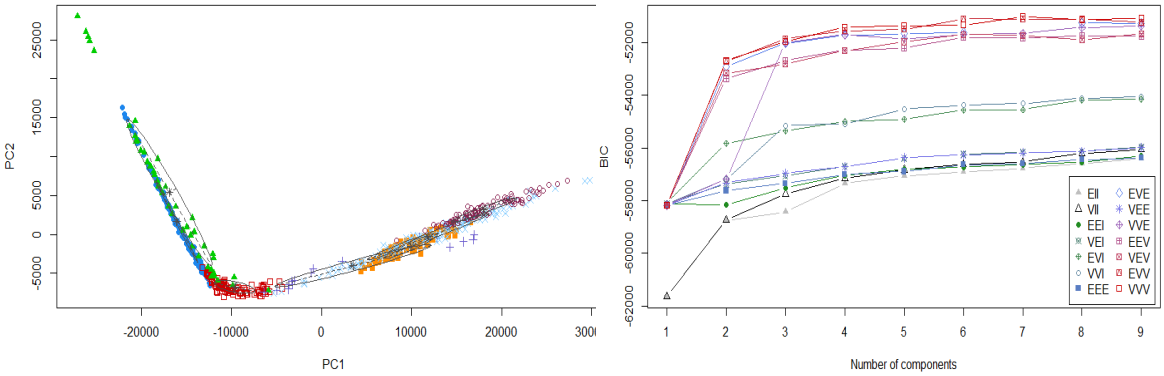


Figure 18: Left: GMM clustering for the first 30 principal components that kept after the implementation of PCA on the extracted features from the FC2. Right: Model selection via BIC criterium.**(Method 3)**

For our third method, the seven prototypes were extracted, and we can see them in Figure 19. The *prototypes images* for the first three clusters (1-3) were actual COVID cases, while the *prototypes images* for the last four clusters (4-7) were actual non-COVID cases.

The mean images for the seven clusters were constructed as we can see in Figure 20. Figure 20 is not as straightforward as Figure 17 because, at first sight, we cannot clearly understand which of the mean images represent a COVID cluster and which a non-COVID one, by just looking to find white patches in the images. The results of the mean images in Figure 20, did not accomplish to inform us so well about the patterns that make an image to be clustered as COVID or not. So, we can say that the mean images of the third method are not seen to be as well-discriminated when compared to the mean images of the second method. Again it is crucial to ask the help of a doctor in order to identify those hidden patterns behind the CT-scans.

One intuitive way that helps to assign a mean label (COVID or non-COVID) to each one of the mean images is to construct a ratio which can be calculated by finding the average weighted sum of the labels ([1,0] for COVID and [0,1] for non-COVID) for all the images. This could be implemented

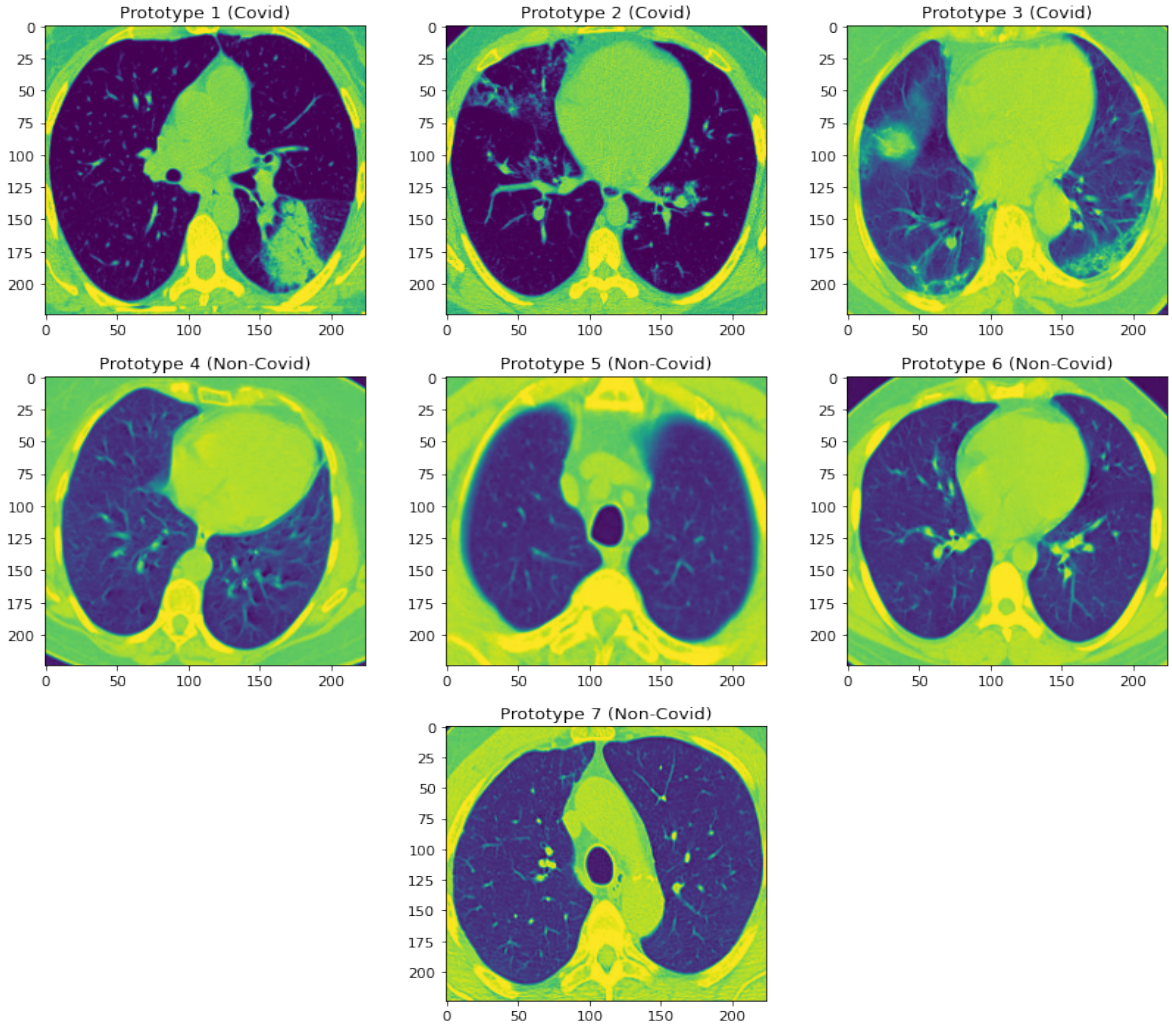


Figure 19: Prototypes images for the seven clusters (Model 3).

by taking a sum of the weighted probabilities over the image labels:

$$Meanlabel_j = \frac{1}{n}\sum_{i=1}^{n} P(image_i \in C_j) \times label_i$$

, where n is the number of images, $Meanlabel_j$ is the mean label of the j_{th} cluster and $label_i$ is the label of the i_{th} image. The term $P(image_i \in C_j)$ represents the clustering assignment of the i_{th} image. The output matrix has $j \times 2$ dimensions, where j is the number of clusters. Essentially, this comparative ratio can provide insights about which one of the two labels (COVID, non-COVID) is more representative for each cluster.

We generated the $Meanlabel_j$ matrix which has 7×2 dimensions, as the generated clusters for the third method were seven. The *mean label* for each *mean image* can be seen in Table 6. As we can see, the first three *mean images* are closer to be COVID, while there are higher chances that the last three *mean images* are non-COVID. This is what we intuitively expected, and it can be clarified with an example. The mean image of one cluster is constructed as the weighted sum of all the images. If the highest weights (probabilities) are coming from the images that have a COVID label, then the COVID

images play a more significant role for this cluster. That is, it can be implied that this mean image is characterised as a "COVID mean image".

One interesting insight we can get from Table 6 is that the 4th mean image has the same chances to be assigned with a COVID and a non-COVID label. This can be explained because a lot of observations of the fourth cluster (purple colour) live at the intersection of the COVID clusters (3 left clusters) and the non-COVID clusters (3 right clusters), as we can see in Figure 18.

	COVID	non-COVID
Mean label 1	0.24	0.01
Mean label 2	0.13	0.01
Mean label 3	0.07	0.01
Mean label 4	0.015	0.015
Mean label 5	0.01	0.12
Mean label 6	0.01	0.21
Mean label 7	0.01	0.14

Table 6: Mean image labels for the seven clusters.

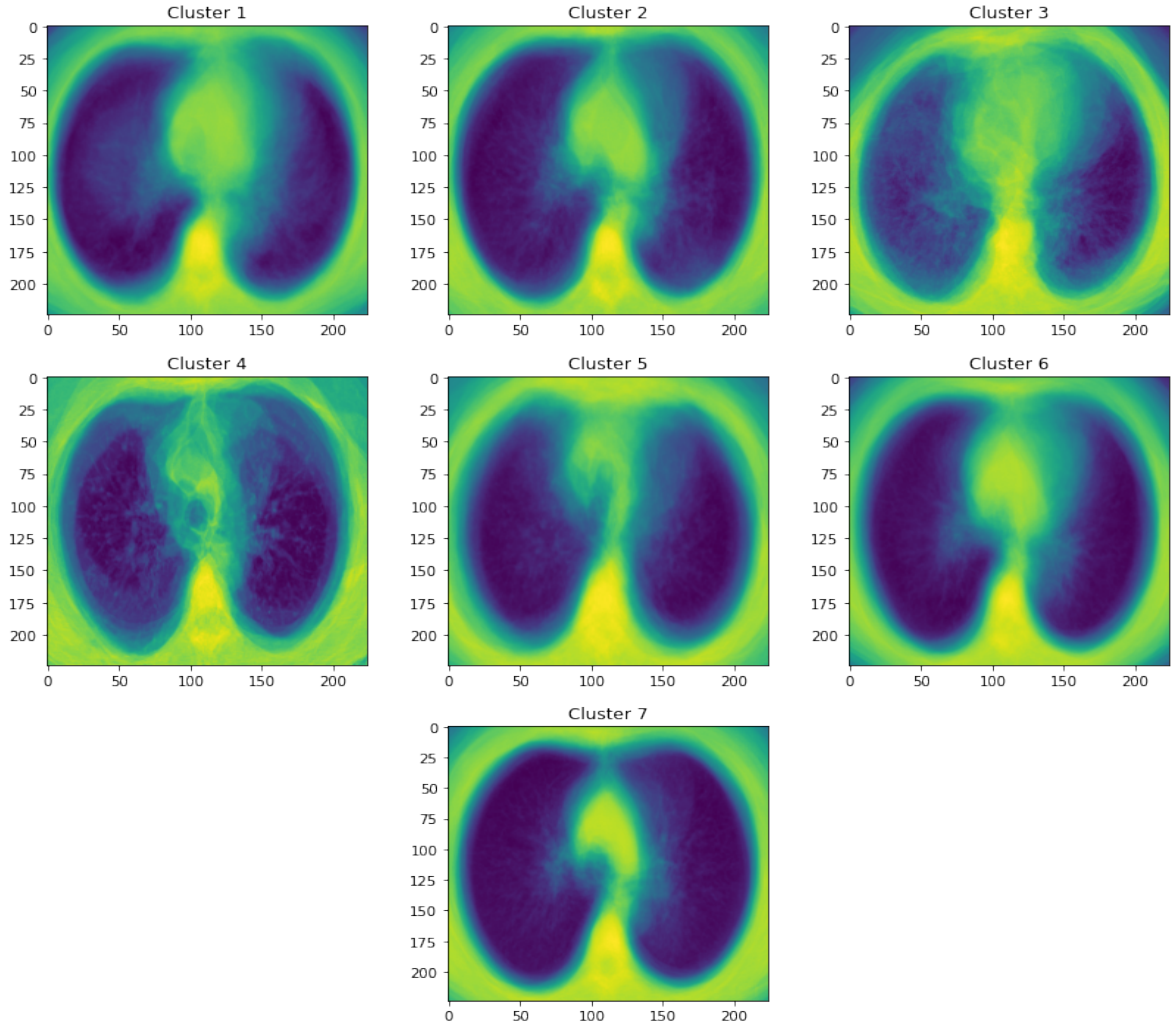


Figure 20: Mean image representation for the seven clusters.

6 Discussion

This research focused on creating a framework which encloses three deep learning models, inspired by the VGG16 architecture, and was combined with a model-based clustering called GMM. GMM was implemented in order to assist medical practitioners in their decision-making. It was shown that unfreezing the last layers can enhance the representativeness of the model and make the model more adaptive to the target problem we aim to solve. Having said that, the last layers trained with our COVID-19 images gradually. To be more specific, according to our first method, only the last layer was trained with our dataset (COVID CT-scans) while at the second and third methods more layers were trained with our training dataset. This is considered a way to obtain better knowledge and to end up with results that are more compact and insightful regarding the interpretability.

The different models were trained, making use of the same settings. That is, categorical cross-entropy used as the loss function and ADAM was selected as the optimiser. Our dataset consists of 1300 images totally and using cross-validation, before generalising on our test data, was taken into account.

Our results were very promising both in terms of performance and explainability. To underpin

the previous argument, all three models accomplished to generalise very well on unseen data and the third model predicted with accuracy 99%. Fine-tuning the models accomplished to boost the model performance while it helps the models to stabilise their performance and to converge quicker.

Generally, ConvNets are widely used as feature extractors, and their outputs are used as classifiers. Nex, we kept the second and third model to continue our analysis because they achieved better performance than the first model. Dimensionality reduction techniques applied to the extracted feature of second and third approaches, the latter can be explained with only the five first principal components covering the ninety per cent (90%) of the total variance, while the former needs more principal components to reach the same amount of variance. So, it was proven that dimensionality reduction using PCA is more successful for our third model because the 90% of the total variance was explained by the first five principal components, and this matches the research of [Oquab et al., 2014] who experimentally showed that mid-layer features are better at transferring knowledge from one big dataset to a different domain dataset with smaller size.

Incorporating model-based clustering techniques to our framework followed next. GMM was implemented on the low-dimensional space produced by our second and third model, and the generating clusters were 6 and 7, respectively, for the two methods. By utilising the soft-clustering property of GMM's, we accomplished to assess each cluster. To achieve that, we assumed that it is more possible for an image to be clustered to the component for which the probability (cluster assignment) is the biggest one. Again, the results were satisfactory as the second method accomplished to cluster correctly the 560 out of 598 images, while the third method succeed better results as it clustered 580 out of 598 images correctly.

Another crucial step that enabled us to get a better insight regarding the clusters was to find the most representative images of each cluster (prototypes) and to compare them with the mean images of each cluster. We constructed the mean image by calculating the weighted sum of the images for each cluster. The weights were represented by the cluster assignments of each image. Potentially, those constructed images would assist the medical researchers and practitioners in understanding which are the specific patterns that may imply that someone is a COVID patient or not.

Predicting on a different dataset also was implemented Specifically, the UCSD dataset which introduced by Yang et al. [2020] was used for this reason, but the results were not so satisfying as the second model achieved to predict with 64% accuracy. The main reason for this may be that the presented models were quite sensitive to the distribution of the images that was fitted to. So we can infer that our models were highly influenced by the assumptions that we made regarding the way that the dataset was distributed. It is worth mentioning that there is no strong evidence that we achieved the goal of external validity, mainly because we only used two different datasets to test our models.

Also, some contributory factors that may cause the discrepancy in performance regarding the testing of the model on the totally new data may be the position of the images or the different sliding techniques that were used on the new dataset. This can be considered as an internal limitation of our dataset because we did not provide a framework that is robust to changes on the position or the angle of the image.

7 Conclusion

To conclude, Deep Learning is a growing field that has gained prominence in the last decade. Data democratisation enabled free access to an enormous amount of available data. In this thesis, it was of high priority to explore different techniques that have been adopted by experts in the field and to examine how we could use them in our problem. One crucial goal of this work was to understand the layer structure in convolutional neural networks by looking in different state-of-the-art architectures like VGG16. After a thorough investigation, we selected the VGG16 architecture as it was seen to meet the requirements of our task. Hence, the model we used as a baseline is the VGG16 which was initially pre-trained with the ImageNet dataset.

Next, the three different modelling strategies were presented, and transfer learning brought quite impressive results in performance without spending a lot of computational resources. Each modelling strategy encloses two different variations; one without fine-tuning and the other with fine-tuning. Fine-tuning managed to enhance the performance of all models and to make them more compatible with the target problem. The third adopted model (Mid-layers Transfer Learning with VGG16) outperformed the baseline accuracy by 5% and recall by 6%. So we can say that the first aim and its objective was met.

The other aim of this work was to explain the image classification decisions using Gaussian Mixture Models. To achieve this, it is crucial to find the low dimensional representations of the raw images first. PCA let us reduce the dimensionality of our dataset. PCA was implemented in all methods and the features of the third method reduced to a 598×5 dimensional matrix while explaining the 90% of the total variance. Next, we implemented model-based clustering to the low-dimensional matrix that was extracted after the dimensionality reduction. The generated clusters of our third proposed model succeed to cluster all the non-COVID cases correctly, and only 19 COVID images were incorrectly assigned to a non-COVID cluster. Next, the most representative images of each cluster (prototypes) were compared to the mean image of each cluster. The prototypes were selected by finding the images that had the lowest Euclidean distance from each cluster centre. The mean images were constructed by using the cluster assignment of the different images.

7.1 Future work

Throughout this thesis, we showed that implementing deep neural network models in the medical field yielded good results, and they can be considered as a trustworthy tool for medical practitioners. For future work, modifications in the model architecture could be examined because selecting a model architecture depends on the nature of the problem. Having said that, the U-Net Convolutional Networks is a famous architecture that can solve a lot of medical imaging problems [Ronneberger et al., 2015]. One important reason to adopt the transfer learning notion is that there is a shortage of available medical images. U-Net can provide an alternative to transfer learning because it requires only a small amount of medical data.

For some future work, data augmentation techniques could be implemented in order to make the model learn some different variations regarding the position of the images. This may force the model to adapt better on new tasks. Also, data augmentation might alleviate the bias, which is introduced to our framework by developing models prone to the distribution of Sars-Cov-2 dataset.

Finally, the main research questions of this study have been seen to be answered. The three proposed models provided good results, but I feel that the superior model is the third one because it has better classification performance. Also, this method accomplished better results on the clustering task. This was seen by the generated clusters of our third method, which better reflected the ideal cluster assignment of our images.

References

Clinical characteristics of covid-19. https://www.ecdc.europa.eu/en/covid-19/latest-evidence/clinical. Accessed: 2020-08-30.

M. Ackermann, S. E. Verleden, M. Kuehnel, A. Haverich, T. Welte, F. Laenger, A. Vanstapel, C. Werlein, H. Stark, A. Tzankov, W. W. Li, V. W. Li, S. J. Mentzer, and D. Jonigk. Pulmonary Vascular Endothelialitis, Thrombosis, and Angiogenesis in Covid-19. *The New England journal of medicine*, 383(2):120–128, jul 2020. ISSN 1533-4406 (Electronic). doi: 10.1056/NEJMoa2015432.

A. Alimadadi, S. Aryal, I. Manandhar, X. P. B. Munroe, and B. Joe. AI and Machine Learning for Understanding Biological Processes Artificial intelligence and machine learning to fight COVID-19. pages 200–202, 2020. doi: 10.1152/physiolgenomics.00029.2020.

Y. Bar, I. Diamant, L. Wolf, S. Lieberman, E. Konen, and H. Greenspan. Chest pathology detection using deep learning with non-medical training. *Proceedings - International Symposium on Biomedical Imaging*, 2015-July:294–297, 2015. ISSN 19458452. doi: 10.1109/ISBI.2015.7163871.

C. M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer-Verlag, Berlin, Heidelberg, 2006. ISBN 0387310738.

C. Bouveyron, G. Celeux, T. B. Murphy, and A. E. Raftery. *Model-Based Clustering and Classification for Data Science: With Applications in R*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 2019. doi: 10.1017/9781108644181.

V. Buhrmester, D. Münch, and M. Arens. Analysis of explainers of black box deep neural networks for computer vision: A survey. *ArXiv*, abs/1911.12116, 2019.

G. Carneiro, J. Nascimento, and A. P. Bradley. Unregistered multiview mammogram analysis with pre-trained deep learning models. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 9351:652–660, 2015. ISSN 16113349. doi: 10.1007/978-3-319-24574-4_78.

G. Csurka, C. Dance, L. Fan, J. Willamowski, and C. Bray. Visual categorization with bags of keypoints. *Work Stat Learn Comput Vision, ECCV*, Vol. 1, 2004.

A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum Likelihood from Incomplete Data Via the EM Algorithm . *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1):1–22, 1977. doi: 10.1111/j.2517-6161.1977.tb01600.x.

Eduardo Soares, Plamen Angelov, Sarah Biaso, Michele Higa Froes, and Daniel Kanda Abe. SARS-CoV-2 CT-scan dataset: A large dataset of real patients CT scans for SARS-CoV-2 identification. *medRxiv*, page 2020.04.24.20078584, 2020. doi: 10.1101/2020.04.24.20078584. URL http://medrxiv.org/content/early/2020/04/29/2020.04.24.20078584.abstract.

M. Ferguson, R. Ak, Y.-T. Lee, and K. Law. Automatic localization of casting defects with convolutional neural networks. pages 1726–1735, 2017. doi: 10.1109/BigData.2017.8258115.

K. Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4):193–202, 1980. ISSN 1432-0770. doi: 10.1007/BF00344251. URL https://doi.org/10.1007/BF00344251.

A. G., T. Kasar, and A. G. Ramakrishnan. A novel GMM based approach for Prototype selection. 2005. doi: 10.13140/RG.2.1.3970.9289.

A. G. Ivakhnenko, V. G. Lapa, and R. Mcdonough. Cybernetics and forecasting techniques. 1967.

A. K. Jain, M. N. Murty, and P. J. Flynn. Data clustering: A review. *ACM Comput. Surv.*, 31(3): 264–323, Sept. 1999. ISSN 0360-0300. doi: 10.1145/331499.331504. URL `https://doi.org/10.1145/331499.331504`.

A. Z. Khuzani, M. Heidari, and S. A. Shariati. COVID-Classifier: An automated machine learning model to assist in the diagnosis of COVID-19 infection in chest x-ray images., may 2020.

D. P. Kingma and J. L. Ba. A : a m s o. pages 1–15, 2015.

A. Krizhevsky, I. Sutskever, and G. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. *Neural Information Processing Systems*, 25, 2012. doi: 10.1145/3065386.

Y. Lecun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. ISSN 14764687. doi: 10.1038/nature14539.

G. Lindsay. Convolutional Neural Networks as a Model of the Visual System: Past, Present, and Future. *Journal of Cognitive Neuroscience*, pages 1–15, 2020. ISSN 0898-929X. doi: 10.1162/jocn_a_01544.

R. Livni, S. Shalev-Shwartz, and O. Shamir. On the computational efficiency of training neural networks. *Advances in Neural Information Processing Systems*, 1(January):855–863, 2014. ISSN 10495258.

D. Masters and C. Luschi. Revisiting small batch training for deep neural networks. *CoRR*, abs/1804.07612, 2018. URL `http://arxiv.org/abs/1804.07612`.

H. Moon and P. J. Phillips. Computational and Performance Aspects of PCA-Based Face-Recognition Algorithms. *Perception*, 30(3):303–321, mar 2001. ISSN 0301-0066. doi: 10.1068/p2896. URL `https://doi.org/10.1068/p2896`.

J. Y. Ng, F. Yang, and L. S. Davis. Exploiting local features from deep networks for image retrieval. In *2015 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 53–61, June 2015. doi: 10.1109/CVPRW.2015.7301272.

T. Nguyen. Artificial Intelligence in the Battle against Coronavirus (COVID-19): A Survey and Future Research Directions. 2020. doi: 10.13140/RG.2.2.36491.23846/1.

K. Nogueira, O. A. Penatti, and J. A. dos Santos. Towards better exploiting convolutional neural networks for remote sensing scene classification. *Pattern Recognition*, 61:539–556, 2017. ISSN 00313203. doi: 10.1016/j.patcog.2016.07.001.

M. Oquab, L. Bottou, I. Laptev, and J. Sivic. Learning and transferring mid-level image representations using convolutional neural networks. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pages 1717–1724, 2014. ISSN 10636919. doi: 10.1109/CVPR.2014.222.

R. Pryor, C. Atkinson, K. Cooper, M. Doll, E. Godbout, M. P. Stevens, and G. Bearman. The electronic medical record and COVID-19: Is it up to the challenge? *American Journal of Infection Control*, 48(8):966–967, 2020. ISSN 0196-6553. doi: https://doi.org/10.1016/j.ajic.2020.05.002. URL `http://www.sciencedirect.com/science/article/pii/S0196655320302741`.

W. Rawat and Z. Wang. Deep Convolutional Neural Networks for Image Classification: A Comprehensive Review. *Neural Computation*, 29:1–98, 2017. doi: 10.1162/NECO_a_00990.

A. S. Razavian, H. Azizpour, J. Sullivan, and S. Carlsson. CNN features off-the-shelf: An astounding baseline for recognition. *IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, pages 512–519, 2014. ISSN 21607516. doi: 10.1109/CVPRW.2014.131.

H. Robbins. A stochastic approximation method. *Annals of Mathematical Statistics*, 22:400–407, 2007.

O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 9351:234–241, 2015. ISSN 16113349. doi: 10.1007/978-3-319-24574-4_28.

S. Ruder. An overview of gradient descent optimization algorithms. pages 1–14, 2016. URL `http://arxiv.org/abs/1609.04747`.

O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, C. V. Jan, J. Krause, and S. Ma. ImageNet Large Scale Visual Recognition Challenge.

L. Scrucca, M. Fop, T. B. Murphy, and A. E. Raftery. mclust 5: Clustering, Classification and Density Estimation Using Gaussian Finite Mixture Models. *The R journal*, 8(1):289–317, aug 2016. ISSN 2073-4859 (Print).

K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings*, pages 1–14, 2015.

C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, 2015. doi: 10.1109/CVPR.2015.7298594.

N. Tajbakhsh, J. Y. Shin, S. R. Gurudu, R. T. Hurst, C. B. Kendall, M. B. Gotway, and J. Liang. Convolutional Neural Networks for Medical Image Analysis: Full Training or Fine Tuning? *IEEE Transactions on Medical Imaging*, 35(5):1299–1312, 2016. ISSN 1558254X. doi: 10.1109/TMI.2016.2535302.

R. T. Trevor Hastie and J. Friedman. *The Elements of Statistical Learning.* Springer New York Inc., 2001.

R. Vaishya, M. Javaid, I. H. Khan, and A. Haleem. Artificial Intelligence (AI) applications for COVID-19 pandemic. *Diabetes and Metabolic Syndrome: Clinical Research and Reviews*, 14(4):337–339, 2020. ISSN 18780334. doi: 10.1016/j.dsx.2020.04.012.

Y. Xie and D. Richmond. Pre-training on grayscale imagenet improves medical image classification. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 11134 LNCS(2):476–484, 2019. ISSN 16113349. doi: 10.1007/978-3-030-11024-6_37.

Y. Xiong and K. Pulli. Color and luminance compensation for mobile panorama construction. pages 261–270, 2010. doi: 10.1145/1873951.1873989.

R. Yamashita, M. Nishio, R. K. G. Do, and K. Togashi. Convolutional neural networks: an overview and application in radiology. *Insights into Imaging*, 9(4):611–629, 2018. ISSN 1869-4101. doi: 10.1007/s13244-018-0639-9. URL `https://doi.org/10.1007/s13244-018-0639-9`.

X. Yang, X. He, J. Zhao, Y. Zhang, S. Zhang, and P. Xie. COVID-CT-Dataset: A CT Scan Dataset about COVID-19. pages 1–14, 2020. URL `http://arxiv.org/abs/2003.13865`.

J. Yosinski, J. Clune, Y. Bengio, and H. Lipson. How transferable are features in deep neural networks? *Advances in Neural Information Processing Systems*, 4(January):3320–3328, 2014. ISSN 10495258.

8 Appendix A

8.1 Project specification

Motivation

Using Deep Learning techniques in the medical sector is a popular topic for data scientists. Deep learning has been proven to provide good results in discovering internal structures in the medical sector. Some domains where Deep learning is used are breast cancer detection, the brain disease classification, and the pneumonia detection from a chest X-ray image. The specific interest of this work is to implement Deep Learning techniques under the context of medical imaging and to proceed from a fundamental general discussion about the breakthroughs of Medical Artificial Intelligence to the practical implementation of Convolutional Neural Networks. This research area is gaining popularity every single day nowadays after the explosion of the Covid-19 pandemic, so the changes that are taking place are dramatic. Hence, this work focuses on pointing out the great achievements that have been done since June 2020 through comparisons of different state-of-the-art algorithms and the evaluation of them in terms of efficiency and explainability.

Data

The dataset that will be used for this project is retrieved from Kaggle. Special interest will be given to the SARS-COV-2 Ct-Scan Dataset which consists of 1252 CT scans from patients that have SARS-CoV-2 virus and 1230 CT scans of non-infected patients. This dataset is almost perfectly balanced, and the images are at greyscale. The images have a different shape, and for that reason, they will be scaled to have the same size.

Aims

Modern deep learning techniques can find practical usage in a broad range of applications. Convolutional neural networks are regularised versions of multilayer perceptron (MLP). The first aim is to construct a model which can analyse COVID-19 CT scan images and predict new COVID patients using their Scans. The second goal of this work is to use transfer learning and to fine-tune these networks for the particular image dataset we are interested in. The third goal is to examine the joint distribution of the feature space produced by the neural network transformation.

1) Create a framework with different model architectures and experiment with them.

The goal is to introduce to explain how Neural Networks work. Next, some different pre-trained algorithms, such as VGG16, will be examined and their structure will be described. Generally, a CNN takes as an input the raw images, and next, there are convolutional and activations layers, pooling layers, and fully-connected layers. Objectives:

- Select the most appropriate model for the examined dataset.
- Explain the theory behind Neural Networks.
- Find the more suitable pre-trained model given the nature of the SARS-COV-2 CtScan Dataset.
- Compare them in terms of computational efficiency and their ability to explain the general structure of the model.

2) Transfer Learning with Deep Neural Networks

In this section, I will evaluate the previous procedure, and appropriate modifications will be implemented. The approach which is suggested uses the learning procedure from the already pre-trained

networks which have been trained using a lot of images comparing to our dataset. That is, it uses the weights to convert the image data into a one-dimensional feature vector. Due to the limited datasets in the medical sector, it is suggestable to use pre-trained models such as ImageNet to accomplish the convergence of weights. This approach is called "Transfer Learning" and it has some advantages when the dataset we want to explore is relatively smaller than the one that was used to build the pre-trained model. This leads the neural network to obtain some prior knowledge from the first pre-trained layers, which has to do with low-level features and to transfer this knowledge to the next features. Fine-tuning is usually useful when it comes to the improvement of the performance. The replacement of some of the last layers with our new SoftMax layer will be considered. Also, learning rate selection and initialisation methods of the weights are important aspects of the modelling procedure, and for this reason, different experimentations will be applied.

Objectives:

- Explore different transfer learning algorithms and experiment with them.
- Through cross-validation, consider how fine tuning will improve the performance of the models.
- Describe the training procedure and evaluate model performance.

3) Understanding Neural Network Activation

Explaining the decisions made by neural network classifiers is an important issue in modern data science. As such, we will try to understand how labels align with the clusters identified in the low dimensional representation provided by the neural network. This can be done by implementing dimensionality reduction techniques (PCA) and clustering (GMM). The explainable results will help doctors to improve their decision-making mechanism by incorporating an efficient tool which can detect and explain the COVID-19 cases.

It is worth to mention that the third aim of this work will be dependent on time constraints, while it constitutes a relatively unexplored topic in the literature.

Deliverables

The Results will be presented, and the evaluation of different algorithms will be shown in a table format (F1, accuracy, precision etc). The different datasets will produce different metrics, so my main goal is to find out which approach has better generalization ability.